

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2026

Egor Razboinikov

Univerzita Pardubice
Fakulta Elektrotechniky a Informatiky

Digitalizace evidence pracovní doby

Bakalářská práce

Univerzita Pardubice
Fakulta elektrotechniky a informatiky
Akademický rok: 2024/2025

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Egor Razboinikov
Osobní číslo:	I21219
Studijní program:	B0688A140009 Informační technologie
Téma práce:	Digitalizace evidence pracovní doby
Zadávací katedra:	Katedra informačních technologií

Zásady pro vypracování

Dokument *evidence pracovní doby* vyučujícího vyžaduje provázanost rozvrhu s úvazkem vyučujícího při respektování času přestávek. Cílem bakalářské práce je navrhnout, vytvořit, odzkoušet a předat funkční multiplatformní aplikaci jejíž vstupy jsou: rozvrh vyučujícího, čas na oběd, plán dovolených, státní svátky, školní volna a další. Požadovaným výstupem je PDF dokument (tabulka) se všemi náležitostmi potřebnými pro odevzdání. Grafické rozhraní poskytne pohodlné zadávání, umožní dodatečné ruční modifikace a zobrazování náhledu před vygenerováním PDF dokumentu.

Rozsah pracovní zprávy: **min. 30 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

PŠENČÍKOVÁ, J. Algoritmizace. Kralice na Hané, Computer Media, 2009.
MAREŠ, Martin a VALLA, Tomáš. Průvodce labyrintem algoritmů 2. vydání. Praha: CZ.NIC, 2022. ISBN 978-80-88168-63-8
PECINOVSKÝ, Rudolf. OOP: naučte se myslet a programovat objektově. Brno: Computer Press, 2010. ISBN 978-80-251-2126-9.

Vedoucí bakalářské práce: **Ing. Radek Matoušek**
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**
Termín odevzdání bakalářské práce: **16. května 2025**

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

L.S.

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

Prohlašuji:

Práci s názvem Digitalizace evidence pracovní doby jsem vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2019 Pravidla pro odevzdávání, zveřejňování a formální úpravu závěrečných prací, ve znění pozdějších dodatků, bude práce zveřejněna prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 9. 5. 2026

Egor Razboinikov v.r.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu své bakalářské práce Ing. Radku Matouškovi za odborné vedení, cenné připomínky a ochotu při konzultacích během zpracování této práce. Poděkování rovněž patří mé rodině a přátelům za podporu po celou dobu studia.

ANOTACE

Cílem této bakalářské práce je navrhnout, implementovat a ověřit multiplatformní desktopovou aplikaci pro digitalizaci evidence pracovní doby vyučujících. Aplikace umožňuje načtení rozvrhových dat ze systému STAG, jejich další zpracování a úpravu v grafickém uživatelském rozhraní. Při zpracování zohledňuje polední přestávky, plánované nepřítomnosti, státní svátky a pravidla evidence pracovní doby. Výstupem aplikace je přehledný dokument ve formátu PDF, který odpovídá požadavkům na odevzdání. Práce se zabývá analýzou problematiky, návrhem architektury a technologií, implementací klíčových částí systému a ověřením výsledného řešení pomocí testování.

KLÍČOVÁ SLOVA

evidence pracovní doby, digitalizace, aplikace pro osobní počítače, C#, .NET, Avalonia UI, MVVM, SQLite, Entity Framework Core, STAG, generování PDF, testování aplikace.

TITLE

Digitization of Working Time Records

ANNOTATION

The aim of this bachelor thesis is to design, implement, and validate a cross-platform desktop application for digitizing teachers' working time records. The application enables the import of timetable data from the STAG system, their further processing, and manual adjustment through a graphical user interface. During processing, it takes into account lunch breaks, planned absences, public holidays, and the rules of working time records. The output of the application is a structured PDF document corresponding to submission requirements. The thesis deals with the analysis of the problem domain, the design of the architecture and technologies, the implementation of key system components, and the validation of the resulting solution through testing.

KEYWORDS

working time records, digitization, desktop application, C#, .NET, Avalonia UI, MVVM, SQLite, Entity Framework Core, STAG, PDF generation, application testing.

OBSAH

SEZNAM ILUSTRACÍ.....	11
SEZNAM ZKRATEK A ZNAČEK.....	12
TERMINOLOGIE	13
ÚVOD.....	14
1 TEORETICKÁ ČÁST	15
1.1 Evidence pracovní doby	15
1.2 Analýza současných přístupů k evidenci pracovní doby	16
1.3 Požadavky na softwarové řešení.....	17
1.3.1 Funkční požadavky	17
1.3.2 Kvalitativní požadavky	18
1.3.3 Shrnutí požadavků	19
1.4 Softwarová architektura a návrhové vzory	20
1.4.1 Softwarová architektura	20
1.4.2 Návrhový vzor MVVM	21
1.4.3 Srovnání MVVM a MVC	22
1.4.4 Monolitická, modulární a mikroslužbová architektura.....	23
1.4.5 Zdůvodnění zvoleného řešení	24
1.5 Použité technologie.....	24
1.5.1 Platforma .NET a jazyk C#	25
1.5.2 Databázová vrstva: SQLite a jazyk SQL	25
1.5.3 Framework Avalonia UI.....	26
1.5.4 Podpůrné knihovny	27
1.5.5 Shrnutí použitých technologií.....	27
1.6 Vývojové prostředí a podpůrné nástroje.....	28
1.6.1 Microsoft Visual Studio	28
1.6.2 SQLiteStudio	29

1.6.3 Shrnutí vývojového prostředí a podpůrných nástrojů.....	29
2 PRAKTICKÁ ČÁST	30
2.1 Návrh řešení.....	30
2.2 Struktura aplikace	31
2.3 Adresářová struktura projektu	32
2.3.1 Prezentační vrstva.....	32
2.3.2 Datová a aplikační vrstva	33
2.3.3 Kořenové soubory projektu	34
2.4 Struktura databáze	34
2.4.1 Přehled tabulek	35
2.4.2 Shrnutí databázového návrhu	37
2.5 Uživatelské rozhraní aplikace.....	38
2.5.1 Hlavní stránka aplikace	38
2.5.2 Okno pro vytvoření události.....	38
2.5.3 Okno pro změnu události.....	39
2.5.4 Vizuální prezentace pro den.....	39
2.5.5 Vizuální prezentace pro týden	40
2.5.6 Vizuální prezentace pro měsíc	40
2.5.7 Globální nastavení	40
2.5.8 Ukázka PDF.....	41
2.5.9 Shrnutí kapitoly	41
2.6 Algoritmy.....	41
2.6.1 Algoritmus vyrovnávání hodin	42
2.6.2 Algoritmus načtení rozvrhu ze systému STAG.....	44
2.7 Testování aplikace	45
2.7.1 Cíl a kritéria přijetí.....	46
2.7.2 Testovací prostředí.....	46

2.7.3 Způsob ověřování	46
2.7.4 Rozdíly mezi platformami	47
2.7.5 Ověřované oblasti testování.....	47
2.7.6 Souhrn ověřených scénářů	48
2.7.7 Shrnutí testování a známá omezení	48
ZÁVĚR.....	49
POUŽITÁ LITERATURA	50

SEZNAM ILUSTRACÍ

Obrázek 1 Vztah mezi částmi View, ViewModel a Model v architektuře MVVM	22
Obrázek 2 Architektura MVC	23
Obrázek 3 Ukázka třídy ApplicationDbContext	33
Obrázek 4 Relační schéma databáze	35
Obrázek 5 Dialogové okno pro vytvoření události	39

SEZNAM ZKRATEK A ZNAČEK

EPD – evidence pracovní doby

PDF – Portable Document Format

SQL – Structured Query Language

UI – User Interface

MVVM – Model–View–ViewModel

MVC – Model–View–Controller

ISO/IEC – International Organization for Standardization / International Electrotechnical Commission

TERMINOLOGIE

Avalonia UI – Multiplatformní framework pro tvorbu uživatelských rozhraní v .NET s deklarativním popisem obrazovek v AXAML. Umožňuje jednotný vzhled a chování na Windows, Linuxu i macOS.

Framework – Sada knihoven, nástrojů a doporučených postupů, která dává aplikaci strukturu a opakovaně použitelné stavební bloky.

Relační databáze – Datový model založený na tabulkách a vztazích (vazby 1:1, 1:N, N:M) s důrazem na integritu a efektivní dotazování.

MVVM (Model–View–ViewModel) – Vzor, který odděluje data a logiku (Model), prezentační logiku (ViewModel) a zobrazení (View). Zlepšuje testovatelnost a udržitelnost UI.

Uživatelské rozhraní (UI) – Vizuální a interaktivní část aplikace (okna, prvky, šablony), v této práci vytvořená v Avalonia UI/AXAML.

IDE – Integrované vývojové prostředí (editor, debugger, správce závislostí a verzí) pro efektivní vývoj.

VCS – Systémy pro správu verzí (např. Git) pro sledování změn a týmovou spolupráci.

STAG – Studijní agenda, zdroj rozvrhových dat pro načtení do aplikace.

EPD – Evidence pracovní doby, oficiální výkaz obsahující měsíční a týdenní souhrny podle interních pravidel.

ÚVOD

Evidence pracovní doby představuje důležitou součást personální a administrativní agendy v různých typech organizací. Slouží k zaznamenávání odpracovaného času zaměstnanců, kontrole plnění pracovních povinností a jako podklad pro další související procesy. Správné vedení evidence pracovní doby je významné nejen z hlediska pracovněprávních požadavků, ale také z hlediska přehlednosti, správnosti a efektivity administrativního zpracování údajů.

V praxi může být vedení evidence pracovní doby spojeno s řadou obtíží. Ty se týkají zejména ručního zadávání údajů, nejednotnosti používaných nástrojů, zvýšené chybovosti a časové náročnosti při zpracování nebo kontrole záznamů. Tyto problémy se mohou dále prohlubovat zejména v prostředí, kde je pracovní režim méně pravidelný a kde je nutné zohledňovat větší množství vstupních údajů, změn a výjimek.

Cílem této bakalářské práce je navrhnout, implementovat a ověřit softwarové řešení, které usnadní evidenci pracovní doby a přispěje ke zpřehlednění celého procesu. Navržená aplikace má umožnit evidenci a úpravu záznamů pracovní doby, zpracování vstupních údajů a vytvoření přehledného výstupu pro další administrativní využití.

Teoretická část práce se nejprve zabývá vymezením evidence pracovní doby a analýzou současných přístupů k jejímu vedení. Dále jsou formulovány požadavky na navržené softwarové řešení a jsou popsány architektonické a technologické prostředky vhodné pro jeho realizaci.

Praktická část práce je věnována návrhu, implementaci a ověření vytvořené aplikace. Zaměřuje se na strukturu aplikace, databázový model, uživatelské rozhraní, algoritmy pro zpracování dat a testování výsledného řešení. Přínosem práce je návrh a realizace aplikace, která může přispět ke zjednodušení evidence pracovní doby a ke snížení administrativní náročnosti tohoto procesu.

1 TEORETICKÁ ČÁST

1.1 Evidence pracovní doby

Evidence pracovní doby představuje významný nástroj pracovněprávní i administrativní praxe. Jejím účelem je zachycovat průběh pracovní doby zaměstnance tak, aby bylo možné kontrolovat dodržování zákonných limitů, vyhodnocovat rozsah odvedené práce a vytvářet podklady pro navazující personální a mzdové procesy. Povinnost vést evidenci pracovní doby ukládá zaměstnavateli zákoník práce, konkrétně § 96, přičemž tato povinnost slouží zejména ke kontrole pracovní doby, doby odpočinku, práce přesčas a dalších skutečností souvisejících s výkonem práce [1, 2].

Z hlediska právní úpravy musí být evidence vedena alespoň v takovém rozsahu, aby z ní byl patrný začátek a konec odpracované směny, odpracovaná práce přesčas, odpracovaná noční práce, odpracovaná doba v době pracovní pohotovosti a pracovní pohotovost, kterou zaměstnanec držel. V širší organizační praxi však evidence pracovní doby obvykle zahrnuje také další údaje důležité pro celkové vyhodnocení pracovního dne, například interně sledované přestávky, nepřítomnosti nebo jiné skutečnosti, které mají význam pro správné posouzení průběhu práce a pro následné administrativní zpracování [2].

Evidence pracovní doby úzce souvisí také s rozvržením pracovní doby. Zaměstnavatel určuje začátek a konec směn a zajišťuje rozvržení pracovní doby, vůči němuž lze následně porovnávat skutečně odpracovaný čas. Právě rozdíl mezi rozvrženou a skutečně odpracovanou dobou je v praxi podstatný například při posuzování práce přesčas, nedodržení plánovaného rozsahu práce nebo jiných odchylek od předpokládaného pracovního režimu [1, 3]. Z tohoto pohledu evidence neplní pouze roli pasivního záznamu, ale představuje i nástroj průběžné kontroly pracovního režimu.

V prostředí vysokých škol má evidence pracovní doby určitá specifika. Akademická činnost se totiž neskládá pouze z přímé výuky, ale také z činností tvůrčích, výzkumných, administrativních a organizačních. Ze zákonného vymezení role vysokých škol i akademických pracovníků lze dovodit, že pracovní režim v tomto prostředí bývá méně pravidelný než v provozech s pevně stanovenými směnami. Přesná evidence pracovní doby tak může být náročnější nejen z hlediska organizace práce, ale také z hlediska vhodně zvoleného technického řešení [4].

Požadavek na kvalitní evidenci pracovní doby potvrzuje i judikatura Soudního dvora Evropské unie, podle níž mají členské státy zajistit, aby zaměstnavatelé používali objektivní, spolehlivý a přístupný systém umožňující měřit délku denní pracovní doby každého pracovníka. Pro praxi

to znamená, že evidence pracovní doby by neměla být pouze formální povinností, ale měla by být vedena přehledně, konzistentně a způsobem, který umožní její následnou kontrolu i ověření správnosti zaznamenaných údajů [5].

1.2 Analýza současných přístupů k evidenci pracovní doby

Způsob vedení evidence pracovní doby se v praxi mezi jednotlivými organizacemi liší. Přestože právní povinnost sledovat pracovní dobu zůstává stejná, konkrétní forma evidence závisí na charakteru organizace, míře digitalizace, složitosti pracovních režimů i na požadavcích na další zpracování údajů. V obecné rovině lze rozlišit tři základní přístupy: ruční evidenci, evidenci prostřednictvím tabulkových nástrojů a evidenci pomocí specializovaných informačních systémů.

Nejjednodušší formu představuje ruční evidence, například v papírové podobě nebo prostřednictvím jednoduchých formulářů. Její výhodou je minimální technická náročnost a snadná dostupnost i v prostředích, kde nejsou k dispozici specializované nástroje. Z hlediska dlouhodobého používání však tento přístup vykazuje řadu omezení. Ruční zapisování zvyšuje pravděpodobnost chyb, komplikuje následnou kontrolu a znesnadňuje vyhledávání, sdílení i archivaci údajů. Současně hůře naplňuje požadavek na objektivní a spolehlivý systém evidence, který je v současné praxi považován za důležitý zejména z hlediska kontroly a prokazatelnosti údajů [5].

Častým mezistupněm mezi ruční a plně automatizovanou evidencí je využití tabulkových nástrojů. Ty umožňují vytvářet vlastní šablony, provádět základní výpočty a připravovat souhrnné přehledy. Výhodou tohoto řešení je relativní flexibilita a nízká pořizovací náročnost. Současně však zůstává značná část procesu závislá na ručním zadávání údajů a na správném nastavení použitých vzorců. Při vyšší variabilitě pracovního režimu nebo při potřebě pravidelných oprav, kontrol a verzování dat se mohou tabulkové nástroje stávat méně přehlednými a méně odolnými vůči chybám. Tento problém se zvyrazňuje zejména v prostředích, kde se pracovní činnost neřídí jednotným a opakujícím se režimem.

Vyspělejší variantu představují specializované informační systémy určené pro evidenci pracovní doby. Jejich přínosem bývá centralizované ukládání dat, automatizace vybraných operací, vyšší míra kontroly správnosti vstupů a možnost propojení s dalšími personálními nebo mzdovými procesy. Takové systémy lépe odpovídají požadavku na přehlednou, ověřitelnou a dostupnou evidenci a mohou významně snížit administrativní zátěž. Na druhou stranu s sebou zpravidla nesou vyšší technickou, organizační a někdy i ekonomickou náročnost. Nevýhodou

může být také menší pružnost v situacích, kdy je třeba systém přizpůsobit specifickým potřebám určité organizace nebo typu práce [6].

Při porovnání jednotlivých přístupů je zřejmé, že se liší zejména mírou automatizace, přehledností, kontrolovatelností a odolností vůči chybám. Ruční a tabulkové formy evidence jsou sice dostupné a snadno použitelné, avšak při pravidelném používání mohou zvyšovat časovou náročnost i riziko nepřesností. Specializované systémy naopak přinášejí vyšší úroveň standardizace a automatizace, avšak jejich přizpůsobení konkrétním pracovním podmínkám může být omezené. V prostředí, kde se pracovní režim vyznačuje vyšší mírou autonomie, proměnlivostí nebo kombinací více druhů činností, se proto jako vhodné jeví takové řešení, které spojí přehlednost, jednoduché zadávání dat a automatizaci pouze tam, kde skutečně přináší praktický přínos [6, 7].

Z uvedené analýzy vyplývá, že vhodné řešení evidence pracovní doby by mělo být srozumitelné, přehledné a snadno použitelné, současně však dostatečně spolehlivé a pružné. Mělo by omezovat chybovost při práci s údaji, podporovat jejich následné vyhodnocení a zjednodušovat administrativní část celého procesu. Tyto závěry tvoří východisko pro formulaci požadavků na vlastní softwarové řešení v následující kapitole.

1.3 Požadavky na softwarové řešení

Na základě vymezení evidence pracovní doby a analýzy současných přístupů lze formulovat požadavky na navrhované softwarové řešení. V oblasti softwarového inženýrství představují požadavky základ pro návrh systému, protože vymezují, co má systém poskytovat, jaké potřeby má naplnit a jaké kvalitativní vlastnosti má vykazovat [8]. Pro účely této práce je vhodné rozlišit požadavky funkční a kvalitativní.

1.3.1 Funkční požadavky

Funkční požadavky určují, jaké činnosti má systém vykonávat a jaké služby má uživateli poskytovat. Základním funkčním požadavkem navrhovaného řešení je práce se záznamy pracovní doby. Systém by měl umožňovat evidovat údaje vztahující se k jednotlivým pracovním dnům, zejména začátek a konec práce, přestávky, typ pracovního dne, případně další údaje, které jsou pro vyhodnocení evidence významné. Tyto údaje musí být vedeny tak, aby bylo možné získat přehled o průběhu práce a následně je využít pro administrativní zpracování [1, 2].

Důležitým požadavkem je rovněž možnost následné úpravy zadaných údajů. V praxi může docházet ke změnám rozvrhu práce, k doplnění chybějících informací nebo k opravám již

vytvořených záznamů. Navržený systém by proto měl umožňovat editaci záznamů tak, aby nebylo nutné vytvářet celý výkaz znovu. Tato vlastnost zvyšuje použitelnost systému a lépe odpovídá reálnému průběhu administrativního zpracování pracovních údajů.

Dalším funkčním požadavkem je zpracování vstupních dat, zejména načtení rozvrhu ze souboru CSV exportovaného ze systému STAG. Aplikace by měla být schopna načíst rozvrhové údaje, ověřit jejich základní správnost a převést je na události uložené v databázi. Tyto události pak tvoří základ pro další práci s evidencí pracovní doby, například pro úpravy v kalendáři, výpočet odpracovaného času, doplnění přestávek a vytvoření výsledného přehledu ve formátu PDF. Tento požadavek je důležitý proto, že bez této funkce by bylo nutné jednotlivé rozvrhové položky zadávat ručně, což by zvyšovalo časovou náročnost i riziko chyb.

Mezi funkční požadavky patří také generování výstupů. Cílem systému nemá být pouze interní uchování údajů, ale i vytvoření přehledného výstupu, který bude možné využít pro další administrativní nebo kontrolní účely. Výstup by měl mít srozumitelnou a standardizovanou podobu a měl by umožňovat jednoduchou interpretaci zaznamenaných údajů. V kontextu této práce je důležité zejména to, aby aplikace dokázala převést uložená data do podoby, kterou lze dále používat mimo samotné uživatelské rozhraní aplikace.

1.3.2 Kvalitativní požadavky

Vedle funkčních požadavků je nutné vymezit také kvalitativní požadavky. Ty nepopisují konkrétní funkce systému, například import dat nebo vytvoření PDF výstupu, ale určují, jaké vlastnosti má aplikace při používání mít. U softwarového řešení pro evidenci pracovní doby je důležité nejen to, aby aplikace požadované činnosti umožňovala, ale také to, aby je prováděla přehledně, spolehlivě a způsobem, který nebude pro uživatele zbytečně složitý. Model kvality ISO/IEC 25010 pracuje mimo jiné s charakteristikami, jako jsou použitelnost, spolehlivost a udržitelnost. Tyto vlastnosti jsou pro navrhované řešení důležité, protože ovlivňují jeho praktickou využitelnost i možnost dalšího rozvoje [9].

Prvním důležitým kvalitativním požadavkem je použitelnost systému, která v této práci úzce souvisí s přehledností uživatelského rozhraní. Uživatel by měl být schopen rychle se orientovat v jednotlivých záznamech, porozumět jejich struktuře a provádět běžné operace bez zbytečné náročnosti. V případě evidence pracovní doby je tento požadavek zvláště důležitý, protože uživatel často pracuje s podobnými údaji, například se začátkem a koncem pracovní doby, přestávkami, typy událostí nebo měsíčními součty. Pokud by bylo rozhraní nepřehledné, mohlo

by docházet k chybám při zadávání, kontrole i následné opravě záznamů. Přehledné rozhraní proto přispívá nejen k pohodlnější práci s aplikací, ale také ke správnosti výsledné evidence.

Druhým významným kvalitativním požadavkem je spolehlivost systému. V této práci se tím rozumí především to, že aplikace musí správně pracovat s uloženými záznamy pracovní doby a musí z nich vytvářet správné výsledky. Pokud uživatel načte stejný rozvrh nebo provede stejnou úpravu dat, aplikace by měla vytvořit stejný výsledek. Zároveň musí bezpečně reagovat na chybné nebo neúplné vstupy, například na neplatný soubor CSV, chybně zadaný čas nebo neúplný záznam, aby nedošlo k vytvoření nekonzistentních dat.

Spolehlivost je důležitá proto, že výstupy aplikace slouží jako podklad pro kontrolu pracovní doby. Chyba při uložení události, výpočtu odpracovaných hodin, určení přestávky nebo vytvoření PDF výstupu by se mohla projevit v celém výsledném přehledu. Navrhované řešení proto musí uchovávat data konzistentně, správně provádět výpočty a umožnit uživateli ověřit výsledný stav evidence.

Třetím důležitým kvalitativním požadavkem je udržitelnost. Tento požadavek se týká především vnitřní struktury aplikace a jejího dalšího vývoje. Navrhované řešení by nemělo být technicky zbytečně složité, ale mělo by být rozděleno do přehledných částí, které mají jasně vymezenou odpovědnost. Takové členění usnadňuje orientaci ve zdrojovém kódu a zároveň snižuje riziko, že změna v jedné části aplikace negativně ovlivní jiné části systému.

Udržitelnost je důležitá zejména v situaci, kdy je nutné aplikaci dále upravovat, opravovat nebo rozšiřovat. Pokud je systém navržen přehledně, lze snáze doplnit nové typy událostí, upravit pravidla výpočtu pracovní doby nebo rozšířit výstupní dokumenty. Udržitelný návrh tak podporuje další rozvoj aplikace bez nutnosti zásadně přepracovat celé řešení.

Na použitelnost, spolehlivost a udržitelnost navazuje také požadavek na omezení chybovosti. V běžné praxi vznikají chyby zejména při ručním zadávání, přepisování údajů nebo následném vyhodnocování pracovní doby. Navrhované řešení by proto mělo tato rizika snižovat například přehlednou strukturou dat, kontrolou vstupních hodnot a automatizací opakujících se kroků. Smyslem aplikace není nahradit veškeré rozhodování uživatele, ale omezit ruční a opakující se úkony, které jsou náchylné k chybám. Díky tomu může být práce s evidencí pracovní doby jednodušší, přehlednější a spolehlivější.

1.3.3 Shrnutí požadavků

Z uvedených požadavků vyplývá, že navrhované softwarové řešení má podporovat hlavní činnosti spojené s evidencí pracovní doby a současně musí být použitelné v běžné praxi.

Aplikace by měla umožňovat vytváření a úpravu záznamů, načtení rozvrhových dat, jejich další zpracování a generování přehledného výstupu. Zároveň by měla být přehledná, spolehlivá, snadno použitelná a navržena tak, aby omezovala vznik chyb při běžné práci.

Takto stanovené požadavky určují, jaké vlastnosti a funkce má navrhovaná aplikace mít. Na jejich základě je možné dále zvolit vhodnou architekturu, rozdělení aplikace do jednotlivých částí a návrhové principy, které budou použity při implementaci. Následující kapitola se proto zaměřuje na softwarovou architekturu a návrhové vzory vhodné pro realizaci navrženého řešení.

1.4 Softwarová architektura a návrhové vzory

Na základě požadavků formulovaných v předchozí kapitole je třeba stanovit takový architektonický přístup, který umožní navrhnout přehledné, udržitelné a spolehlivé softwarové řešení. Volba vhodné softwarové architektury ovlivňuje nejen vnitřní strukturu aplikace, ale také způsob zpracování dat, oddělení odpovědností mezi jednotlivými částmi systému a možnosti jeho dalšího rozvoje. V případě aplikace zaměřené na evidenci pracovní doby je důležité zejména to, aby architektura podporovala přehlednou práci s daty, snadné úpravy jednotlivých částí systému a jednoznačné oddělení uživatelského rozhraní od aplikační logiky.

1.4.1 Softwarová architektura

Softwarovou architekturu lze chápat jako soubor základních rozhodnutí o struktuře systému, jeho hlavních komponentách a vztazích mezi nimi. Určuje, z jakých částí se aplikace skládá, jaké odpovědnosti jednotlivé části nesou a jakým způsobem spolu komunikují. Správně navržená architektura přispívá ke srozumitelnosti systému, usnadňuje jeho údržbu a umožňuje efektivnější rozšiřování funkcí v budoucnu [10, 11].

V kontextu této práce má volba architektury význam především proto, že navržené řešení pracuje s více typy činností, například se zpracováním vstupních dat, jejich úpravou, vyhodnocením a tvorbou výstupů. Pokud by tyto části nebyly dostatečně odděleny, mohlo by docházet ke zhoršení přehlednosti kódu, vyšší provázanosti jednotlivých komponent a obtížnější testovatelnosti aplikace. Z tohoto důvodu je vhodné zvolit takový architektonický přístup, který umožní rozdělit systém na logicky oddělené části a zároveň zachovat jednoduchost celého řešení.

1.4.2 Návrhový vzor MVVM

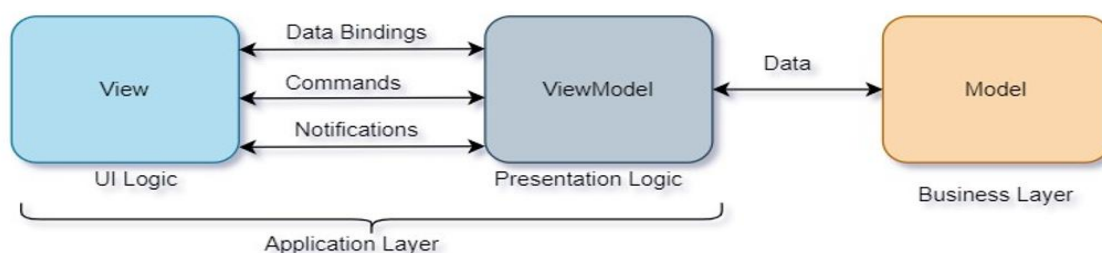
Jedním z vhodných návrhových vzorů pro aplikace s grafickým uživatelským rozhraním je MVVM, tedy *Model–View–ViewModel*. Tento přístup rozděluje aplikaci do tří základních částí: *View*, *ViewModel* a *Model*. *View* představuje uživatelské rozhraní, tedy obrazovky, dialogová okna a ovládací prvky, se kterými uživatel pracuje. *ViewModel* tvoří prostřední vrstvu mezi uživatelským rozhraním a modelem. Připravuje data pro zobrazení, uchovává stav dané obrazovky a reaguje na uživatelské akce. Model představuje datovou a aplikační část systému, tedy objekty a pravidla, se kterými aplikace pracuje [12, 13].

Vztah mezi těmito částmi je znázorněn na *obrázku 1*. *View* komunikuje s *ViewModelem* především pomocí datových vazeb, příkazů a oznámení. Díky tomu nemusí uživatelské rozhraní přímo obsahovat logiku zpracování dat. *ViewModel* zprostředkovává komunikaci mezi rozhraním a modelem a převádí data do podoby vhodné pro zobrazení uživateli.

V kontextu této práce *Model* nepředstavuje pouze jednu třídu ani přímé napojení na databázi. Jde spíše o část aplikace, která zahrnuje datové objekty, pravidla pro práci s nimi a návaznost na služby zajišťující ukládání, načítání a zpracování údajů. Samotná práce s databází, načtení rozvrhu nebo výpočty pracovní doby jsou v aplikaci řešeny v samostatných službách a datové vrstvě. *ViewModel* tyto části využívá a jejich výsledky poskytuje uživatelskému rozhraní.

Význam MVVM spočívá především v oddělení uživatelského rozhraní od logiky aplikace. *View* se stará o vzhled a rozložení prvků, zatímco *ViewModel* určuje, jaká data se mají zobrazit a jak má aplikace reagovat na uživatelské vstupy. Tento přístup podporuje přehlednější organizaci kódu, protože jednotlivé části aplikace mají jasněji vymezenou odpovědnost. Zároveň usnadňuje úpravy uživatelského rozhraní, protože změna vzhledu nemusí znamenat zásah do zpracování dat.

Pro desktopové aplikace využívající deklarativní způsob tvorby rozhraní je MVVM přirozeným a často používaným přístupem. Umožňuje efektivně pracovat se stavem aplikace, datovými vazbami mezi *View* a *ViewModelem* a reakcemi na uživatelské akce. Z hlediska požadavků formulovaných v předchozí kapitole je tento vzor vhodný zejména proto, že podporuje přehlednost, udržitelnost a jednoznačné rozdělení odpovědností [12].



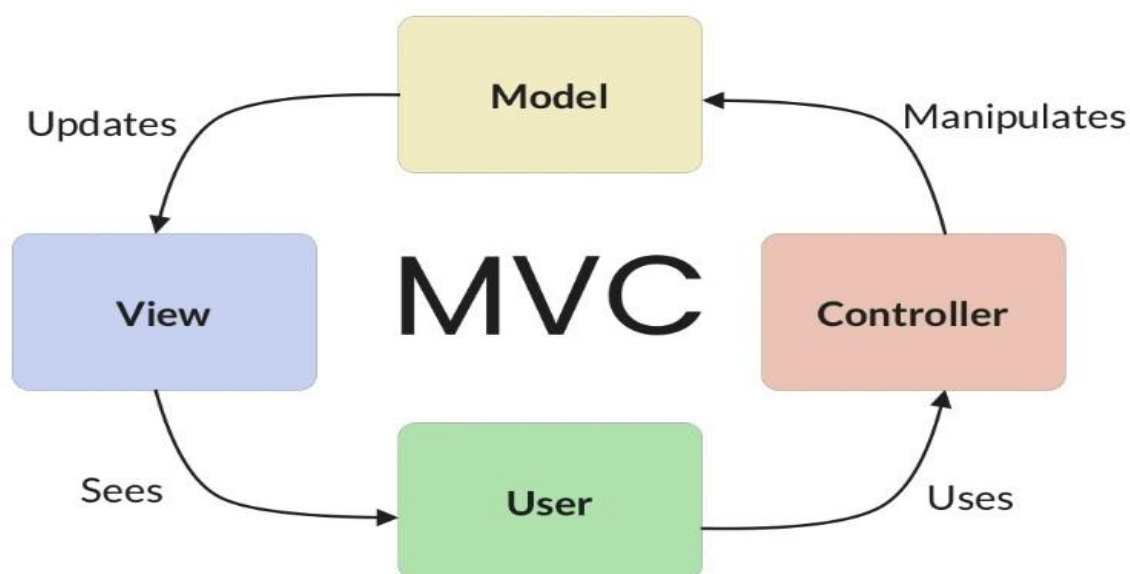
Obrázek 1 Vztah mezi částmi View, ViewModel a Model v architektuře MVVM

1.4.3 Srovnání MVVM a MVC

Vedle vzoru MVVM se v softwarovém inženýrství často používá také návrhový vzor MVC, tedy *Model–View–Controller*. Stejně jako MVVM i tento přístup usiluje o oddělení jednotlivých odpovědností v aplikaci. *Model* představuje data a logiku, *View* slouží k jejich zobrazení a *Controller* zajišťuje zpracování vstupů od uživatele a řízení toku aplikace [14].

MVC je tradičně spojován zejména s aplikacemi, ve kterých *Controller* přímo reaguje na akce uživatele a rozhoduje o tom, jaká data budou zobrazena. Tento přístup je vhodný především v prostředích, kde je potřeba explicitně řídit komunikaci mezi uživatelským vstupem a výstupní vrstvou. V případě moderních desktopových aplikací s deklarativním uživatelským rozhraním však bývá vhodnější MVVM, protože lépe odpovídá principu datových vazeb a oddělení prezentační logiky od samotného zobrazení.

Zatímco MVC klade důraz na řídicí roli *Controlleru*, MVVM více využívá zprostředkující vrstvu *ViewModelu*, která připravuje data pro prezentaci a omezuje přímou závislost mezi rozhraním a logikou aplikace. Pro aplikaci zaměřenou na evidenci pracovní doby je tento přístup vhodnější zejména proto, že podporuje přehlednější práci se stavem formulářů, záznamů a výstupních údajů. *ViewModel* tak umožňuje soustředit prezentační logiku do samostatné vrstvy a zachovat samotné zobrazení jednodušší a lépe upravitelné.



Obrázek 2 Architektura MVC

1.4.4 Monolitická, modulární a mikroslužbová architektura

Při návrhu softwarového řešení je kromě volby návrhového vzoru důležité také zvolit celkové uspořádání aplikace. Jednou z možností je monolitická architektura, při níž je systém vytvářen jako jeden celek. Výhodou takového přístupu je jednoduchost vývoje, nasazení i správy, protože aplikace tvoří jedinou spustitelnou jednotku. Nevýhodou však může být menší přehlednost při růstu systému a obtížnější oddělení jednotlivých částí aplikace [15].

Určitou variantu tohoto přístupu představuje modulární monolit. I v tomto případě jde o jednu aplikaci, její vnitřní struktura je však rozdělena do samostatných modulů s jasně vymezenými odpovědnostmi. Takové řešení umožňuje zachovat jednoduchost nasazení a současně zlepšit organizaci systému, jeho přehlednost a udržitelnost. Jednotlivé části aplikace mohou být navrženy tak, aby každá řešila konkrétní oblast funkcionality, aniž by docházelo k jejich zbytečnému provázání [16].

Další možností je mikroslužbová architektura, která rozděluje systém na více samostatně provozovaných služeb. Tento přístup může být výhodný u rozsáhlých systémů s vysokými nároky na škálování, samostatné nasazování jednotlivých částí nebo oddělenou správu více týmů. Současně však přináší vyšší technickou složitost, protože vyžaduje řešení komunikace mezi službami, správu distribuovaného prostředí a větší provozní nároky [17].

Pro aplikaci menšího až středního rozsahu, jejímž cílem je evidence pracovní doby a generování přehledných výstupů, se jako vhodnější jeví jednodušší architektonické řešení. V takovém případě je důležité především zachovat přehlednost, snadnou údržbu a nízkou provozní

náročnost, nikoli budovat distribuovaný systém s vlastnostmi, které pro daný účel nejsou nezbytné. Vlastní aplikace proto nemusí využívat mikroslužbový přístup, ale měla by být navržena tak, aby její části byly uvnitř projektu přehledně oddělené.

1.4.5 Zdůvodnění zvoleného řešení

S ohledem na požadavky uvedené v kapitole 1.3 je pro navrhované řešení vhodné zvolit architekturu, která podporuje jednoduchost, přehlednost a oddělení jednotlivých částí systému. Aplikace určená pro evidenci pracovní doby nevyžaduje složitou distribuovanou infrastrukturu ani samostatně provozované služby. Důležitější je, aby byla snadno použitelná, přehledná a aby její jednotlivé části bylo možné dále upravovat nebo rozšiřovat.

Z tohoto důvodu je vhodným řešením modulární monolit. Tento přístup umožňuje zachovat aplikaci jako jeden celek, ale zároveň ji uvnitř rozdělit do logických částí s jasně vymezenou odpovědností. Takové členění je vhodné zejména pro menší a středně rozsáhlé desktopové aplikace, u nichž je důležitá jednoduchost nasazení, nízká provozní náročnost a dobrá udržitelnost. Modulární uspořádání současně omezuje nežádoucí provázanost jednotlivých částí systému.

Pro prezentační část je vhodné využít návrhový vzor MVVM. Tento vzor podporuje oddělení uživatelského rozhraní od části aplikace, která připravuje data pro zobrazení a zpracovává reakce na uživatelské akce. *View* se zaměřuje na vizuální podobu rozhraní, zatímco *ViewModel* zprostředkovává data a stav potřebný pro jeho fungování. Díky tomu lze uživatelské rozhraní měnit nebo rozšiřovat bez nutnosti zásadně zasahovat do ostatních částí systému.

Výhodou tohoto přístupu je také lepší přehlednost zdrojového kódu a snazší údržba aplikace. Oddělení odpovědností umožňuje samostatněji řešit práci s daty, aplikační pravidla a prezentační vrstvu. To je důležité zejména u aplikace, která pracuje s opakovaně upravovanými záznamy, provádí jejich vyhodnocování a vytváří výstupy pro další použití.

Kombinace modulárního monolitu a návrhového vzoru MVVM proto odpovídá charakteru navrhovaného řešení. Umožňuje zachovat jednoduchost aplikace a současně podporuje požadavky na přehlednost, použitelnost, spolehlivost a udržitelnost. Na základě této architektonické volby lze v další kapitole přistoupit k představení technologií vhodných pro implementaci aplikace.

1.5 Použité technologie

Na základě zvolené architektury a návrhových principů je v dalším kroku vhodné vymezit technologie, které byly pro realizaci navrženého řešení použity. Volba konkrétních technologií

má přímý vliv na způsob implementace aplikace, její přenositelnost, udržitelnost i možnosti dalšího rozvoje. V případě aplikace zaměřené na evidenci pracovní doby je důležité zvolit takové technologické prostředky, které umožní vytvořit přehledné uživatelské rozhraní, spolehlivě zpracovávat data a generovat odpovídající výstupy.

S ohledem na požadavky formulované v předchozích kapitolách byly pro realizaci aplikace zvoleny technologie platformy .NET, programovací jazyk C#, databázový systém SQLite a framework Avalonia UI. Tyto prostředky společně vytvářejí vhodný základ pro tvorbu desktopové aplikace, která je přehledná, snadno rozšiřitelná a současně technicky nenáročná na nasazení.

1.5.1 Platforma .NET a jazyk C#

Platforma .NET představuje moderní vývojové prostředí určené pro tvorbu různých typů aplikací, včetně desktopových, webových i mobilních řešení. Její výhodou je především široká podpora nástrojů, stabilní ekosystém a možnost vytvářet přenositelná řešení pro více operačních systémů. Pro tuto práci je podstatné, že platforma .NET poskytuje prostředí pro vývoj aplikace kombinující práci s daty, grafické uživatelské rozhraní a generování výstupních dokumentů [2].

Programovací jazyk C# byl zvolen především pro svou čitelnost, silnou typovou kontrolu a dobrou podporu objektově orientovaného přístupu. Tyto vlastnosti přispívají k přehlednější struktuře kódu, usnadňují jeho údržbu a snižují riziko některých typů chyb při vývoji. Výhodou C# je rovněž dobrá integrace s ostatními částmi platformy .NET, což umožňuje efektivně propojit jednotlivé vrstvy aplikace do jednoho funkčního celku [1, 2].

Pro navržené řešení je tato technologie vhodná také proto, že podporuje tvorbu aplikací založených na architektonickém vzoru MVVM. Díky tomu je možné lépe oddělit prezentační vrstvu od aplikační logiky, což odpovídá požadavkům na přehlednost, udržitelnost a testovatelnost systému.

1.5.2 Databázová vrstva: SQLite a jazyk SQL

Součástí navrženého řešení je také práce s daty, která je potřeba ukládat, načítat a dále zpracovávat. Pro tento účel byla zvolena databáze SQLite, tedy lehký relační databázový systém vhodný zejména pro lokální ukládání dat v rámci jedné aplikace. Jeho výhodou je jednoduché nasazení, protože nevyžaduje samostatný databázový server ani složitou správu databázového prostředí [20].

SQLite je vhodnou volbou pro aplikace menšího a středního rozsahu, u nichž je kladen důraz na lokální ukládání dat, jednoduché nasazení a nízké provozní nároky. V kontextu této práce

představuje vhodné řešení pro ukládání údajů souvisejících s evidencí pracovní doby, protože umožňuje uchovávat strukturovaná data v přehledné podobě a současně poskytuje dostatečné možnosti pro jejich následné vyhledávání a zpracování.

Databázová vrstva navrhované aplikace není určena pro správu více zaměstnanců ani pro registrační systém. Aplikace je koncipována jako lokální desktopové řešení, ve kterém pracuje jeden uživatel s vlastní evidencí pracovní doby. Databáze proto slouží především k ukládání záznamů pracovních dnů, souvisejících údajů a nastavení potřebných pro správné fungování aplikace. Tento přístup odpovídá charakteru práce, protože cílem není vytvořit personální informační systém pro více uživatelů, ale praktický nástroj pro lokální zpracování evidence.

Při práci s databází je využíván jazyk SQL, který slouží k definici datových struktur i k manipulaci s uloženými daty. SQL umožňuje vytvářet tabulky, upravovat obsah databáze a získávat data podle zadaných podmínek. Pro potřeby této práce je důležitý zejména proto, že poskytuje standardizovaný a srozumitelný způsob práce s relačními daty [21].

1.5.3 Framework Avalonia UI

Pro tvorbu grafického uživatelského rozhraní byl zvolen aplikační framework Avalonia UI. Jedná se o moderní framework pro vývoj desktopových aplikací na platformě .NET, který umožňuje vytvářet multiplatformní uživatelská rozhraní s využitím deklarativního přístupu. To znamená, že vzhled aplikace je možné definovat odděleně od její logiky, což podporuje přehlednost a lepší organizaci celého projektu [12].

Významnou vlastností frameworku Avalonia UI je jeho přirozená podpora architektonického vzoru MVVM. Díky tomu je možné navrhnout uživatelské rozhraní tak, aby bylo odděleno od logiky zpracování dat a jednotlivé vrstvy aplikace nebyly zbytečně provázané. Tento přístup je pro navržené řešení vhodný zejména proto, že aplikace pracuje s větším množstvím údajů, formulářových prvků a výstupních přehledů, u nichž je důležitá přehledná správa stavu a jednoznačné rozdělení odpovědností.

Další výhodou Avalonia UI je možnost vytvářet jednotné uživatelské prostředí napříč více operačními systémy. To odpovídá požadavku na přenositelnost aplikace a současně umožňuje zachovat jednotný způsob ovládání a zobrazení dat bez ohledu na konkrétní platformu. V případě lokální desktopové aplikace je tato vlastnost vhodná také z hlediska případného budoucího rozšíření nebo přenesení řešení na jiné pracovní prostředí.

1.5.4 Podpůrné knihovny

Kromě hlavních technologií byly v projektu využity také podpůrné knihovny, které rozšiřují základní funkčnost aplikace a usnadňují implementaci vybraných úloh. Tyto knihovny nepředstavují samostatné jádro řešení, ale doplňují hlavní technologický celek o prostředky pro efektivnější práci s uživatelským rozhraním, aplikační logikou a tvorbou výstupů.

V souvislosti s architekturou MVVM byla využita knihovna ReactiveUI, která podporuje práci se stavem aplikace, vazbami mezi daty a uživatelským rozhraním a organizací prezentační logiky. Její využití je vhodné zejména v situacích, kdy aplikace pracuje s více formulářovými prvky, změnami hodnot a reakcemi na uživatelské akce. Díky tomu lze část logiky související s prezentací dat soustředit do *ViewModelů* a zachovat přehlednější strukturu projektu [22].

Pro řešení vybraných interakcí v uživatelském rozhraní lze využít také mechanismus Behaviors, který umožňuje oddělit chování rozhraní od samotné vizuální definice prvků. Tento přístup je užitečný například v případech, kdy je potřeba doplnit určité chování ovládacích prvků, aniž by byla narušena přehlednost hlavní struktury uživatelského rozhraní [23].

Pro generování výstupních dokumentů byla využita knihovna QuestPDF. Ta umožňuje v prostředí C# a .NET programově definovat strukturu dokumentu, doplnit tabulky a vytvořit výsledný soubor ve formátu PDF. Knihovna umožňuje programově definovat strukturu dokumentu, doplnit tabulky a vytvořit výsledný soubor ve formátu PDF. V kontextu aplikace pro evidenci pracovní doby je tato možnost důležitá zejména proto, že výsledná data nemají zůstat pouze v databázi, ale mají být dostupná také v podobě samostatného výstupního dokumentu [24].

Pro následnou práci se soubory PDF lze využít nástroje Ghostscript a Ghostscript.NET, které rozšiřují možnosti práce s tímto formátem v prostředí .NET. Tyto nástroje mohou být využitelné zejména v situacích, kdy je potřeba s vygenerovanými dokumenty dále pracovat, ověřovat jejich podobu nebo je připravovat pro další použití [25, 26].

Využití podpůrných knihoven umožňuje soustředit se na vlastní logiku aplikace a současně snižuje potřebu implementovat běžné technické mechanismy od začátku. Při jejich výběru je důležité zohlednit kompatibilitu se zvolenou architekturou, dlouhodobou udržitelnost a vhodnost pro daný typ aplikace.

1.5.5 Shrnutí použitých technologií

Z výše uvedeného přehledu vyplývá, že zvolené technologie vytvářejí vhodný základ pro realizaci navrženého softwarového řešení. Platforma .NET spolu s programovacím jazykem C#

umožňuje vývoj přehledné a udržitelné aplikace. SQLite poskytuje jednoduché prostředí pro lokální správu dat a Avalonia UI podporuje tvorbu uživatelského rozhraní odpovídajícího zvolenému architektonickému přístupu.

Podpůrné knihovny doplňují základní technologické prostředky o další funkce potřebné pro implementaci aplikace. ReactiveUI podporuje práci s prezentační logikou, QuestPDF umožňuje generování výstupních dokumentů a další nástroje rozšiřují možnosti práce s PDF soubory. Celek tak odpovídá požadavkům na aplikaci, která má být lokální, přehledná, technicky nenáročná a dostatečně flexibilní pro zpracování evidence pracovní doby.

Na tuto kapitolu přirozeně navazuje představení vývojového prostředí a dalších nástrojů, které byly při implementaci využity.

1.6 Vývojové prostředí a podpůrné nástroje

Vedle volby vhodné architektury a technologií je pro realizaci softwarového řešení důležité také vývojové prostředí a podpůrné nástroje, které usnadňují implementaci, ladění a správu projektu. Tyto prostředky sice netvoří samotný základ navržené aplikace, ale mají významný vliv na efektivitu vývoje, přehlednost práce se zdrojovým kódem a možnost ověřování správnosti jednotlivých částí systému.

V rámci této práce byly využity nástroje, které podporují vývoj aplikace na platformě .NET, práci s databází SQLite a průběžnou kontrolu vytvářeného řešení. Jejich výběr vychází z požadavku na přehledné, stabilní a opakovatelné vývojové prostředí, které umožňuje efektivně realizovat jednotlivé části navržené aplikace.

1.6.1 Microsoft Visual Studio

Pro vývoj aplikace bylo využito integrované vývojové prostředí Microsoft Visual Studio. Jedná se o nástroj určený pro tvorbu aplikací na platformě .NET, který poskytuje prostředky pro psaní zdrojového kódu, správu projektové struktury, kompilaci, ladění i testování aplikace. Výhodou tohoto prostředí je zejména jeho komplexnost, díky níž lze většinu vývojových činností provádět v jednom sjednoceném rozhraní [27].

Microsoft Visual Studio podporuje práci s jazykem C# a poskytuje funkce, které usnadňují orientaci ve zdrojovém kódu, odhalování chyb a průběžné ověřování správnosti implementace. Důležitou vlastností je také podpora správy externích balíčků a knihoven, které jsou v projektu využívány. Tím je zajištěna lepší přehlednost závislostí a snadnější správa použitých komponent.

V kontextu této práce je Visual Studio vhodným nástrojem také proto, že podporuje vývoj desktopových aplikací na platformě .NET a umožňuje efektivní práci s více částmi projektu, například s logikou aplikace, datovou vrstvou i uživatelským rozhraním. Jeho využití tak přispívá k přehlednosti vývoje a k rychlejšímu ověřování navrženého řešení.

1.6.2 SQLiteStudio

Pro práci s databází byl využit nástroj SQLiteStudio, který slouží ke správě a kontrole databázových souborů systému SQLite. Tento nástroj umožňuje přehledné zobrazení databázové struktury, práci s tabulkami, spouštění SQL dotazů a ověřování uložených dat mimo samotnou aplikaci [28].

Význam SQLiteStudio spočívá především v tom, že usnadňuje kontrolu databázového schématu a obsahu databáze během vývoje. Díky tomu je možné jednoduše ověřovat správnost navržených tabulek, kontrolovat uložené záznamy a testovat databázové operace nezávisle na uživatelském rozhraní aplikace. Tento postup přispívá k lepší přehlednosti vývoje a usnadňuje identifikaci případných chyb při práci s daty.

V rámci této práce představuje SQLiteStudio vhodný podpůrný nástroj zejména proto, že doplňuje hlavní vývojové prostředí o možnost přímé práce s databází. Tím usnadňuje nejen návrh a kontrolu datové vrstvy, ale také průběžné ověřování správného fungování celé aplikace.

1.6.3 Shrnutí vývojového prostředí a podpůrných nástrojů

Zvolené vývojové prostředí a podpůrné nástroje vytvářejí vhodné podmínky pro realizaci navrženého softwarového řešení. Microsoft Visual Studio poskytuje komplexní prostředí pro implementaci, ladění a správu projektu, zatímco SQLiteStudio umožňuje efektivní kontrolu a správu databázové vrstvy. Jejich kombinace přispívá k přehlednosti vývoje, usnadňuje ověřování jednotlivých částí systému a podporuje stabilní realizaci aplikace.

Na teoretickou část práce navazuje praktická část, která se již zaměřuje na samotný návrh, implementaci a ověření vytvořeného softwarového řešení.

2 PRAKTICKÁ ČÁST

Praktická část této práce se zaměřuje na návrh, implementaci a ověření aplikace určené pro evidenci pracovní doby. Navržené řešení vychází z požadavků formulovaných v teoretické části, zejména z požadavků na práci se záznamy pracovní doby, zpracování vstupních dat, vytvoření přehledného výstupu a omezení chybovosti při běžné práci s údaji. Tato část práce popisuje, jak byly tyto požadavky převedeny do konkrétního návrhu aplikace a jak byly realizovány její hlavní části.

V následujících kapitolách je nejprve popsán návrh řešení a vnitřní struktura aplikace. Dále je uvedena organizace zdrojových souborů, návrh databázové vrstvy a způsob práce s uloženými daty. Následuje popis uživatelského rozhraní, které umožňuje s funkcemi aplikace pracovat. Samostatná část je poté věnována algoritmům pro načtení rozvrhu, vytváření a úpravu pracovních záznamů, doplňování přestávek a vyrovnávání pracovní doby. Závěr praktické části se zaměřuje na testování aplikace a ověření správnosti jejího fungování.

2.1 Návrh řešení

Navržené řešení je desktopová aplikace určená pro lokální zpracování evidence pracovní doby. Aplikace pracuje s údaji o pracovních dnech, událostech, přestávkách, nastavení pracovního režimu a výstupních přehledech. Základním principem návrhu je převést vstupní údaje do jednotné vnitřní struktury, nad kterou lze následně provádět výpočty, kontroly a generování výsledného výstupu.

Důležitou částí návrhu je práce se vstupními daty. Aplikace umožňuje načíst rozvrhové údaje ze souboru CSV a převést je na události uložené v databázi. Tyto události pak tvoří základ pro další zpracování evidence pracovní doby. Součástí návrhu je také možnost ruční úpravy záznamů, protože v praxi může docházet ke změnám pracovního režimu, opravám načtených rozvrhových údajů nebo doplnění dalších událostí.

Na práci s daty navazuje aplikační logika, která zajišťuje vyhodnocení jednotlivých dnů a správné uspořádání záznamů. Významnou roli zde mají algoritmy pro generování automatických pracovních bloků, doplňování obědových přestávek a vyrovnávání pracovní doby. Tyto části aplikace jsou důležité proto, že samotné uložení událostí nestačí. Údaje je nutné dále zpracovat tak, aby odpovídaly pravidlům evidence pracovní doby a bylo možné z nich vytvořit správný měsíční přehled.

Další částí návrhu je vytvoření výstupu ve formátu PDF. Výstupní dokument slouží jako přehled evidence pracovní doby za zvolené období a obsahuje údaje potřebné pro kontrolu odpracovaného času, přestávek, přesčasů a neodpracované doby. Návrh aplikace proto počítá nejen s interním uložením dat, ale také s jejich převodem do podoby, kterou lze použít mimo samotnou aplikaci.

Uživatelské rozhraní tvoří prezentační část aplikace. Jeho úkolem je zpřístupnit hlavní funkce uživateli, zobrazit uložené záznamy v přehledné podobě a umožnit jejich úpravu. Rozhraní tedy navazuje na datovou a aplikační část systému a poskytuje uživateli prostředky pro práci s načtenými rozvrhovými i ručně vytvořenými údaji. Při jeho návrhu byl kladen důraz na srozumitelnost a jednoduché ovládání, aby bylo možné s aplikací pracovat bez zbytečně složitých kroků.

Celkový návrh aplikace je proto rozdělen do několika částí: datové vrstvy, aplikační logiky, algoritmického zpracování, generování výstupů a uživatelského rozhraní. Toto členění umožňuje popsat jednotlivé části řešení samostatně a zároveň ukázat, jak na sebe navazují při běžném zpracování evidence pracovní doby.

2.2 Struktura aplikace

Aplikace je navržena jako desktopové softwarové řešení, které pracuje s lokálně uloženými daty a poskytuje uživateli grafické rozhraní pro správu evidence pracovní doby. Z hlediska vnitřní organizace je rozdělena do několika vzájemně provázaných částí, z nichž každá plní specifickou úlohu. Toto členění vychází ze zvoleného architektonického přístupu a umožňuje oddělit zpracování dat, aplikační logiku a prezentační vrstvu.

Základ systému tvoří datová vrstva, která zajišťuje ukládání a načítání údajů potřebných pro chod aplikace. Na ni navazuje aplikační logika, jejímž úkolem je zpracování vstupních dat, provádění kontrol, vyhodnocování záznamů a příprava údajů pro jejich zobrazení. Samostatnou část představuje vrstva uživatelského rozhraní, která zprostředkovává komunikaci mezi uživatelem a aplikací a umožňuje práci s jednotlivými funkcemi systému.

Důležitou součástí aplikace jsou také podpůrné služby, které zajišťují specifické operace, například načtení rozvrhu, validaci, zpracování vybraných pravidel evidence pracovní doby nebo generování výstupních dokumentů. Tím je dosaženo lepšího oddělení odpovědností mezi jednotlivými částmi systému a současně je usnadněna orientace ve zdrojovém kódu i případné budoucí rozšíření aplikace.

Takto navržená struktura odpovídá požadavku na přehlednost a udržitelnost řešení. Současně vytváří vhodný základ pro implementaci funkcí, které jsou pro evidenci pracovní doby klíčové, a umožňuje návazně popsat organizaci zdrojových souborů, databázovou strukturu a jednotlivé části uživatelského rozhraní.

2.3 Adresářová struktura projektu

Souborová struktura projektu byla navržena tak, aby odpovídala rozdělení aplikace do logických částí a současně podporovala přehlednost zdrojového kódu. Organizace projektu vychází ze zvoleného architektonického přístupu a rozlišuje zejména části určené pro uživatelské rozhraní, aplikační logiku, práci s daty a podpůrné mechanismy. Takové uspořádání usnadňuje orientaci v projektu, zjednodušuje údržbu a vytváří vhodné podmínky pro další rozšiřování aplikace.

Jednotlivé složky projektu sdružují soubory podle jejich funkce v systému. Díky tomu je možné oddělit prezentační vrstvu od datové a aplikační vrstvy a současně zachovat přehledné členění zdrojového kódu. Přehled adresářové struktury projektu je uveden v příloze B.

2.3.1 Prezentační vrstva

Prezentační vrstva zahrnuje části projektu, které zajišťují zobrazení dat, reakci na uživatelské akce a vizuální podobu aplikace. Patří sem zejména složky *Views*, *ViewModels*, *Controls*, *Behaviors* a *Converters*.

Složka *Views* obsahuje jednotlivá zobrazení uživatelského rozhraní, například hlavní okna, dialogy a další obrazovky aplikace. Složka *ViewModels* zahrnuje prvky prezentační logiky, které zprostředkovávají komunikaci mezi datovou vrstvou a uživatelským rozhraním. *ViewModely* připravují data pro zobrazení, reagují na uživatelské akce a řídí chování jednotlivých obrazovek.

Složka *Controls* obsahuje vlastní uživatelské ovládací prvky, které jsou opakovaně využívány v různých částech aplikace. Jejich účelem je sjednotit vzhled a chování vybraných částí rozhraní a omezit duplicitu v definici uživatelských prvků. Složka *Behaviors* obsahuje prvky určené pro rozšíření chování uživatelského rozhraní bez nutnosti zasahovat přímo do logiky jednotlivých zobrazení. Tyto komponenty slouží zejména k zachycení nebo úpravě vybraných interakcí mezi uživatelem a aplikací. Složka *Converters* obsahuje převodníky používané při vazbě dat mezi aplikační logikou a uživatelským rozhraním. Jejich úkolem je upravovat datové hodnoty do podoby vhodné pro zobrazení nebo další zpracování v rámci rozhraní aplikace.

2.3.2 Datová a aplikační vrstva

Datová a aplikační vrstva je tvořena zejména složkami *Data*, *Models*, *Services*, *Helpers* a *Messages*. Tyto části projektu zajišťují ukládání dat, jejich zpracování a podporu hlavních funkcí aplikace.

Složka *Data* obsahuje prvky související s databázovou vrstvou aplikace, zejména datový kontext a konfiguraci mapování entit na databázové tabulky. Tato část projektu zajišťuje komunikaci s databází SQLite a podporuje jednotný způsob ukládání a načítání dat.

Složka *Models* zahrnuje datové modely reprezentující základní objekty aplikace. Tyto modely slouží k uchování a předávání údajů, se kterými aplikace pracuje v rámci evidence pracovní doby. Složka *Services* obsahuje služby zajišťující hlavní aplikační operace, například zpracování vstupních dat, validaci, import nebo generování výstupů. Tato vrstva soustřeďuje logiku, která není přímo vázána na uživatelské rozhraní.

Složka *Helpers* obsahuje pomocné třídy a utility, které usnadňují implementaci opakujících se operací. Složka *Messages* slouží k definici zpráv a komunikačních objektů používaných při předávání informací mezi jednotlivými částmi aplikace. Jejím účelem je podpořit oddělení odpovědností a zjednodušit interní komunikaci systému.

```
namespace TeacherScheduleApp.Data
{
    Ссылка: 71 | Egor Razboinikov, 6 дн. назад | Автор: 1, изменений: 7
    public class AppDbContext : DbContext
    {
        Ссылка: 4 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<Employee> Employees { get; set; }
        Ссылка: 7 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<SemesterSettings> SemesterSettings { get; set; }
        Ссылка: 1 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<WeekdaySettings> WeekdaySettings { get; set; }
        Ссылка: 12 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<DaySettings> DaySettings { get; set; }
        Ссылка: 2 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<ImportBatch> ImportBatches { get; set; }
        Ссылка: 52 | Egor Razboinikov, 357 дн. назад | Автор: 1, изменение: 1
        public DbSet<Event> Events { get; set; }
        Ссылка: 11 | Egor Razboinikov, 15 дн. назад | Автор: 1, изменение: 1
        public DbSet<BalanceSelfTrim> BalanceSelfTrims { get; set; }
        Ссылка: 10 | Egor Razboinikov, 15 дн. назад | Автор: 1, изменение: 1
        public DbSet<BalanceTransfer> BalanceTransfers { get; set; }

        Ссылка: 0 | Egor Razboinikov, 6 дн. назад | Автор: 1, изменений: 3
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            if (optionsBuilder.IsConfigured)
                return;

            var appDir = AppContext.BaseDirectory;
            var dbPath = Path.Combine(appDir, "teacherapp.db");

            optionsBuilder.UseSqlite($"Data Source={dbPath}");
        }

        Ссылка: 0 | Egor Razboinikov, 7 дн. назад | Автор: 1, изменений: 6
        protected override void OnModelCreating(ModelBuilder modelBuilder) ...
    }
}
```

Obrázek 3 Ukázka třídy AppDbContext

2.3.3 Kořenové soubory projektu

Kořenové soubory projektu obsahují základní konfigurační, spouštěcí a sestavovací části aplikace. Patří sem zejména soubory potřebné pro inicializaci aplikace, správu hlavních nastavení projektu a zajištění jeho sestavení. Tyto soubory vytvářejí nezbytný základ pro provoz celé aplikace a doplňují její vnitřní strukturu o technické prvky potřebné pro vývoj a spuštění.

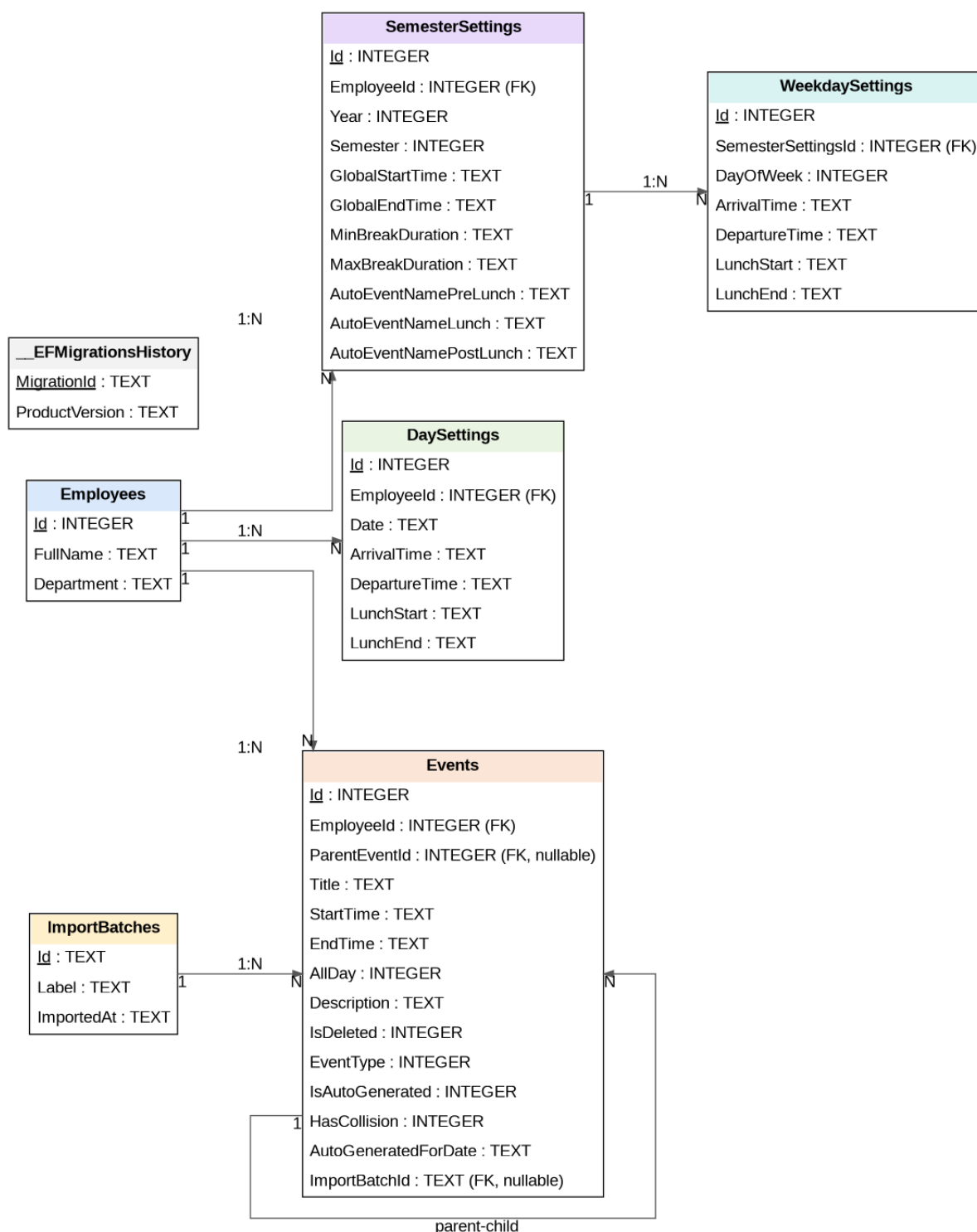
2.4 Struktura databáze

Databázová vrstva aplikace byla navržena jako relační databáze uložená v systému SQLite. Jejím úkolem je uchovávat údaje potřebné pro evidenci pracovní doby jednoho lokálního uživatele, tedy základní identifikační údaje, pracovní události, nastavení pracovního režimu a informace o načtených rozvrhových datech. Databáze je navržena tak, aby jednotlivé části aplikace pracovaly se stejnými daty a aby bylo možné záznamy následně zobrazovat, upravovat, vyhodnocovat a použít při tvorbě výsledného výstupu.

Aplikace není určena jako víceuživatelský personální systém. Neobsahuje registraci uživatelů ani správu seznamu zaměstnanců. Přesto databázový model obsahuje tabulku *Employees*. V této aplikaci tato tabulka nepředstavuje evidenci zaměstnanců v personálním smyslu, ale technický záznam osoby, pro kterou je evidence pracovní doby vedena. Aplikace tedy v běžném použití pracuje s jedním lokálním profilem zaměstnance.

Na tento lokální profil jsou navázány další části databáze. Tabulka *Events* ukládá jednotlivé události v kalendáři, například pracovní bloky, obědové přestávky, výuku, služební cesty nebo nepřítomnosti. Tabulka *DaySettings* uchovává nastavení konkrétního pracovního dne, například začátek a konec pracovní doby nebo čas obědové přestávky. Tabulky *SemesterSettings* a *WeekdaySettings* slouží k uložení výchozího pracovního režimu pro semestr a jednotlivé dny v týdnu. Tabulka *ImportBatches* uchovává informace o dávkách načtených rozvrhových dat. Při načtení rozvrhu ze systému STAG aplikace vytvoří záznam dávky a přiřadí k němu nově vzniklé rozvrhové události.

Vazby mezi tabulkami jsou vytvořeny pomocí cizích klíčů. Díky tomu lze jednoznačně určit, ke kterému lokálnímu profilu patří konkrétní událost, denní nastavení nebo dávka načtených dat. Databázová struktura tím podporuje konzistentní práci s údaji a omezuje riziko, že by se jednotlivé záznamy používané v aplikaci od sebe oddělily. Relační schéma databáze je uvedeno na *obrázku 4*.



Obrázek 4 Relační schéma databáze

2.4.1 Přehled tabulek

Databázový model aplikace je tvořen několika tabulkami, které společně pokrývají hlavní oblasti práce s evidencí pracovní doby. Jedna část databáze uchovává základní údaje o lokálním profilu zaměstnance, další část ukládá časové události a samostatné tabulky slouží pro nastavení pracovního režimu a pro evidenci importovaných dat. Podrobné názvy sloupců a jejich datové

typy jsou znázorněny v relačním schématu databáze na *obrázku 4*. Následující text proto vysvětluje hlavně účel jednotlivých tabulek v aplikaci.

Tabulka *Employees* uchovává základní identifikační údaje osoby, pro kterou je evidence pracovní doby vytvářena. V kontextu této aplikace nejde o seznam zaměstnanců ani o personální databázi. Tabulka slouží jako lokální profil zaměstnance, ke kterému jsou přiřazeny pracovní události, nastavení pracovního režimu a další související údaje. Tyto údaje mohou být využity také při generování výstupního dokumentu.

Tabulka *Events* je hlavní tabulkou pro ukládání časových záznamů. Obsahuje jednotlivé události, se kterými uživatel pracuje v kalendáři aplikace. Do této tabulky se ukládá název události, začátek a konec události, popis, typ události a informace o tom, zda jde o celodenní záznam. Tabulka zároveň obsahuje příznaky, podle kterých aplikace rozlišuje ručně vytvořené události, rozvrhové události a automaticky vytvořené pracovní bloky. Pracovní blok v této práci označuje časový úsek vytvořený aplikací jako část pracovní doby; v denním pohledu je zobrazen modrým blokem, jak je patrné v *příloze E*.

Součástí tabulky *Events* jsou také údaje související se stavem záznamu. Aplikace tak může určit, zda byla událost označena jako smazaná, zda u ní vznikla časová kolize nebo zda patří ke konkrétní importní dávce. Díky tomu lze v jedné tabulce uchovávat různé druhy událostí, ale současně zachovat jejich jasné rozlišení při dalším zpracování.

Tabulka *DaySettings* slouží k uložení nastavení konkrétního dne. Obsahuje zejména začátek a konec pracovní doby a časové nastavení obědové přestávky. Důležitý je také příznak *IsManualOverride*, který určuje, zda bylo nastavení daného dne změněno ručně. Pokud uživatel upraví konkrétní den, aplikace může toto nastavení zachovat a nepřepsat ho běžnými výchozími hodnotami.

Tabulka *SemesterSettings* ukládá obecné nastavení pracovního režimu pro konkrétní semestr. Obsahuje například výchozí názvy automaticky vytvářených pracovních bloků, minimální a maximální délku přestávky a další hodnoty používané při zpracování evidence pracovní doby. Tato tabulka tvoří základní konfiguraci, ze které aplikace vychází při vytváření a úpravě pracovních záznamů.

Tabulka *WeekdaySettings* doplňuje semestrální nastavení o pravidla pro jednotlivé dny v týdnu. Umožňuje nastavit, že například pondělí, úterý nebo jiný pracovní den může mít jiný začátek pracovní doby, jiný konec pracovní doby nebo jiný čas obědové přestávky. Díky tomu aplikace

nemusí používat pro všechny dny stejné hodnoty a může lépe odpovídat skutečnému pracovnímu režimu.

Tabulka *ImportBatches* uchovává informace o importovaných dávkách dat. Při importu rozvrhu ze systému STAG aplikace vytvoří záznam importní dávky a k němu přiřadí nově vzniklé události. Díky tomu je možné později určit, ze kterého importu konkrétní události pocházejí. Tento princip je důležitý také při opakovaném importu stejného období, protože aplikace může starší importované události nahradit aktuálními daty.

Tabulka *__EFMigrationsHistory* je technická tabulka vytvářená nástrojem Entity Framework Core. Neobsahuje aplikační data, se kterými by přímo pracoval uživatel. Slouží k evidenci databázových migrací, tedy změn databázového schématu provedených během vývoje aplikace.

Z uvedeného přehledu vyplývá, že databázový model je zaměřen na praktické potřeby aplikace. Ukládá lokální profil zaměstnance, pracovní události, nastavení pracovního režimu a informace o importech. Tyto tabulky společně vytvářejí základ pro další aplikační logiku, zejména pro zobrazení záznamů v kalendáři, automatické vytváření pracovních bloků, kontrolu obědových přestávek, vyrovnávání pracovní doby a generování výsledného dokumentu.

2.4.2 Shrnutí databázového návrhu

Navržená databázová struktura odpovídá požadavkům aplikace na ukládání údajů o lokálním uživatelském profilu, událostech, nastavení pracovního režimu a importovaných datech. Jednotlivé oblasti dat jsou rozděleny do samostatných tabulek, což přispívá k přehlednosti návrhu, jednodušší správě dat a lepší udržitelnosti celého řešení.

Důležitou vlastností databázového návrhu je také existence vazeb mezi hlavními entitami, které zajišťují referenční integritu a umožňují jednoznačné propojení jednotlivých částí systému. Tyto vazby nejsou určeny pro správu více registrovaných zaměstnanců, ale pro konzistentní propojení pracovních událostí, denních nastavení, semestrálních pravidel a importních dávek s jedním lokálním záznamem osoby, pro kterou je evidence vedena.

Navržená struktura tak vytváří vhodný základ pro implementaci aplikační logiky i pro následnou práci s uživatelským rozhraním aplikace. Současně odpovídá charakteru řešení, které je určeno jako lokální desktopová aplikace bez registračního systému a bez víceuživatelské správy zaměstnanců.

2.5 Uživatelské rozhraní aplikace

Uživatelské rozhraní aplikace bylo navrženo tak, aby uživatel mohl pracovat s evidencí pracovní doby bez nutnosti zasahovat přímo do databáze. Rozhraní proto soustřeďuje hlavní funkce aplikace do několika přehledných částí: správu událostí, načtení rozvrhu, nastavení pracovního režimu, kalendářové zobrazení a náhled výsledného dokumentu.

Důležitou součástí návrhu bylo, aby jednotlivé části aplikace na sebe navazovaly. Uživatel může nejprve načíst rozvrh, následně zkontrolovat nebo upravit vytvořené události, ověřit pracovní dobu v denním, týdenním nebo měsíčním pohledu, a nakonec zobrazit výsledný dokument ve formátu PDF. Rozhraní tak odpovídá běžnému postupu práce s aplikací.

2.5.1 Hlavní stránka aplikace

Hlavní stránka aplikace představuje základní pracovní prostředí. Je rozdělena na levou ovládací část a pravou kalendářovou část. Levá část slouží k ovládání hlavních funkcí aplikace. Uživatel zde může vytvořit novou událost, načíst rozvrh ze souboru, otevřít globální nastavení nebo zobrazit náhled výsledné evidence pracovní doby.

Součástí levé části je také kalendář pro výběr konkrétního dne, přehled načtených souborů a informace o denním nastavení pracovní doby. Tyto prvky umožňují rychle zjistit, s jakým dnem uživatel právě pracuje a jaké základní hodnoty jsou pro daný den použity.

Pravá část hlavní stránky zobrazuje samotnou evidenci pracovní doby v kalendářové podobě. Uživatel zde vidí jednotlivé události a pracovní bloky v čase. Podle potřeby může přepínat mezi denním, týdenním a měsíčním pohledem. Ukázka levé části hlavní stránky je uvedena v příloze C a ukázka pravé části hlavní stránky v příloze D.

2.5.2 Okno pro vytvoření události

Okno pro vytvoření události slouží k zadání nového záznamu do evidence pracovní doby. Uživatel zde vyplňuje název události, volitelný popis, datum, čas začátku, čas konce a typ události. Podle zvoleného typu je následně událost zpracována v dalších částech aplikace.

Okno umožňuje zadat také celodenní událost. V takovém případě uživatel nemusí určovat přesný časový interval, protože záznam se vztahuje k celému dni. Při ukládání aplikace kontroluje povinné údaje a omezuje tak vznik neúplných nebo časově nesprávných záznamů.

Vytvoření události je důležité zejména v situacích, kdy je potřeba doplnit záznam, který nebyl načten z rozvrhu, například nepřítomnost, služební cestu nebo jinou ruční úpravu pracovního dne. Ukázka okna pro vytvoření události je uvedena na *obrázku 5*.

Nová událost

Název

Popis

☐ Celý den

Začátek:

14 duben 2026 8 30

Konec:

14 duben 2026 17 00

Typ události: Práce

Zrušit Vytvořit

Obrázek 5 Dialogové okno pro vytvoření události

2.5.3 Okno pro změnu události

Okno pro změnu události slouží k úpravě již existujícího záznamu. Má stejnou základní strukturu jako okno pro vytvoření nové události, které je uvedeno na *obrázku 5*. Rozdíl je v tom, že při otevření existující události jsou jednotlivá pole již vyplněna aktuálními hodnotami.

Uživatel může změnit název události, popis, datum, časové vymezení nebo typ události. Po potvrzení změny aplikace záznam uloží a následně může přepočítat navazující části evidence pracovní doby, pokud je to pro daný typ úpravy potřeba.

Při úpravě existující události se v okně navíc zobrazuje tlačítko pro odstranění záznamu. Odstranění je odděleno od běžného uložení změn, aby bylo zřejmé, že se jedná o jinou akci. Tím je zachováno jednoduché ovládání a zároveň je možné pracovat se stejným typem okna jak při vytváření, tak při úpravě událostí.

2.5.4 Vizualní prezentace pro den

Denní pohled slouží k podrobné kontrole jednoho konkrétního dne. Zobrazuje časovou osu dne a jednotlivé události umístěné podle času začátku a konce. Díky tomu uživatel vidí, jak jsou v daném dni rozloženy pracovní bloky, výuka, obědová přestávka nebo další záznamy.

Tento pohled je důležitý hlavně při kontrole konkrétních časových údajů. Uživatel může snadno poznat, zda se události překrývají, zda je obědová přestávka umístěna ve správném čase a zda

pracovní bloky odpovídají nastavení dne. Ukázka denního pohledu aplikace je uvedena v *příloze E*.

2.5.5 Vizualní prezentace pro týden

Týdenní pohled zobrazuje více pracovních dnů vedle sebe. Uživatel díky němu získá přehled o rozložení pracovní doby v rámci celého týdne. Tento pohled je vhodný zejména při kontrole návaznosti jednotlivých dnů a při ověřování, zda pracovní doba odpovídá požadovanému týdennímu fondu.

V týdenním pohledu lze sledovat nejen jednotlivé události, ale také jejich rozložení mezi pracovními dny. To je důležité například po importu rozvrhu nebo po ručních úpravách, kdy je potřeba ověřit, zda aplikace správně doplnila pracovní bloky a přestávky. Ukázka týdenního pohledu aplikace je uvedena v *příloze F*.

2.5.6 Vizualní prezentace pro měsíc

Měsíční pohled poskytuje celkový kalendářový přehled za zvolený měsíc. Jednotlivé dny jsou zobrazeny v měsíční mřížce a mohou být vizuálně odlišeny podle svého charakteru. Uživatel tak rychle pozná běžné pracovní dny, víkendy, státní svátky nebo dny patřící do sousedního měsíce.

Toto zobrazení je určeno hlavně pro orientační kontrolu delšího období. Neposkytuje tak podrobný časový detail jako denní pohled, ale umožňuje rychle zkontrolovat, zda jsou v měsíci evidovány očekávané události a zda některé dny nevyžadují další úpravu. Ukázka měsíčního pohledu aplikace je uvedena v *příloze G*.

2.5.7 Globální nastavení

Globální nastavení slouží k úpravě hodnot, ze kterých aplikace vychází při zpracování evidence pracovní doby. Uživatel zde nastavuje zejména časové parametry pracovního dne, obědovou přestávku a názvy automaticky vytvářených pracovních bloků.

Nastavení je možné rozlišit podle období, například podle semestru. To je důležité, protože pracovní režim se může v různých částech roku lišit. Hodnoty z globálního nastavení jsou následně využívány při generování pracovních bloků, kontrole obědových přestávek a přípravě výsledného dokumentu. Ukázka obrazovky globálního nastavení je uvedena v *příloze H*.

2.5.8 Ukázka PDF

Náhled výsledného dokumentu ve formátu PDF je důležitý, protože právě tento dokument představuje finální výstup aplikace. Uživatel si zde může ověřit, jak bude evidence pracovní doby za zvolené období vypadat před uložením do počítače.

V náhledu lze přepínat mezi jednotlivými měsíci a zkontrolovat obsah vytvořené tabulky. Uživatel tak může ještě před uložením ověřit, zda dokument obsahuje správné dny, pracovní dobu, přestávky, přesčasy, neodpracovanou dobu a souhrnné hodnoty. Tato část rozhraní propojuje práci s daty v aplikaci s výsledným dokumentem, který může být dále použit mimo aplikaci. Ukázka výsledného dokumentu evidence pracovní doby je uvedena v *příloze I*.

2.5.9 Shrnutí kapitoly

Uživatelské rozhraní aplikace umožňuje provádět hlavní pracovní kroky potřebné pro vytvoření evidence pracovní doby. Uživatel může načíst rozvrh, vytvořit nebo upravit události, zkontrolovat jejich zobrazení v kalendáři, nastavit pracovní režim a připravit výsledný dokument ve formátu PDF.

Jednotlivé pohledy mají rozdílný účel. Denní pohled slouží k detailní kontrole konkrétního dne, týdenní pohled k ověření rozložení pracovní doby v rámci týdne a měsíční pohled k celkové orientaci v delším období. Náhled PDF potom umožňuje zkontrolovat finální výstup aplikace. Tím rozhraní vytváří praktické prostředí pro běžnou práci s evidencí pracovní doby a navazuje na algoritmy popsané v následující kapitole.

2.6 Algoritmy

Tato kapitola popisuje postupy, které v aplikaci zajišťují zpracování ručně zadaných událostí a událostí převzatých z rozvrhu. Jde zejména o doplnění pracovních bloků, kontrolu obědových přestávek, vyrovnaní pracovní doby a načtení rozvrhových dat ze systému STAG do podoby použitelné v evidenci pracovní doby.

V aplikaci na sebe navazují dvě hlavní části zpracování. První část tvoří vyrovnavání hodin. Tento postup porovnává započtený pracovní čas s požadovaným počtem pracovních hodin a podle výsledku upravuje pouze ty části dne, které aplikace vytvořila automaticky. Ručně zadané události a události převzaté z rozvrhu zůstávají zachovány, protože jde o údaje zadané uživatelem nebo převzaté z externího systému.

Druhou část tvoří načtení rozvrhu ze systému STAG. Aplikace načte soubor ve formátu CSV, zkontroluje jeho obsah a vytvoří z něj rozvrhové události. Rozvrhovou událostí se v této práci rozumí záznam výuky převzatý z rozvrhu vyučujícího, například přednáška, cvičení nebo jiná

časově vymezená výuková činnost. Po uložení těchto událostí aplikace spustí další zpracování, aby se pracovní bloky, obědové přestávky a celková evidence pracovní doby přizpůsobily nově načtenému rozvrhu.

Průběh vybraných částí zpracování je znázorněn v přílohách pomocí blokových schémat. Algoritmus vyrovnávání hodin je z důvodu rozsahu rozdělen do několika navazujících schémat, která popisují celkový průběh vyrovnání, týdenní úpravy a kontrolu obědových přestávek. Načtení rozvrhu ze systému STAG je znázorněno samostatně ve dvou schématech: první schéma zachycuje výběr a kontrolu souboru CSV včetně zpracování jednotlivých řádků, druhé schéma navazuje uložení vytvořených událostí do databáze a následným spuštěním dalších kroků aplikace.

2.6.1 Algoritmus vyrovnávání hodin

Algoritmus vyrovnávání hodin slouží k tomu, aby výsledná evidence pracovní doby odpovídala požadovanému počtu pracovních hodin. V aplikaci však nelze upravovat všechny události stejným způsobem. Události vytvořené uživatelem ručně a události převzaté z rozvrhu se považují za pevné záznamy. Algoritmus je proto nepřepisuje a nemění jejich čas. Úpravy se provádějí především na automaticky vytvořených pracovních blocích, tedy na časových úsecích, které aplikace doplnila jako pracovní dobu.

Nejprve se určí pracovní dny, se kterými má algoritmus pracovat. Do výpočtu se nezahrnují víkendy a státní svátky, protože pro tyto dny se běžný pracovní fond nepočítá. Vyrovnávání může být spuštěno pro konkrétní týden, celý měsíc nebo rozsah dnů, který se změnil například po načtení rozvrhu nebo po ruční úpravě události.

Základní výpočet probíhá po týdnech. Pro každý pracovní den aplikace zjistí, kolik času je možné započítat jako práci. Do započteného času patří pracovní bloky a rozvrhové události, zatímco obědová přestávka se jako práce nepočítá. Následně se stav celého týdne porovná s požadovaným počtem pracovních hodin. Pokud je v týdnu hodin více, vzniká přebytek. Pokud hodiny chybí, vzniká nedostatek.

Při přebytku hodin se aplikace snaží zkrátit automaticky vytvořené pracovní bloky. Nejprve hledá části dne, které lze upravit bez zásahu do ručně zadaných událostí nebo událostí převzatých z rozvrhu. Zkrácení se provádí hlavně na začátku nebo na konci pracovního dne. Pokud konkrétní den obsahuje pevné události na obou okrajích dne, není vhodné ho jednoduše zkrátit. V takovém případě se přebytek řeší pomocí jiných dnů ve stejném týdnu, kde lze automatický pracovní blok upravit bezpečněji.

Při nedostatku hodin aplikace postupuje opačně. Snaží se doplnit chybějící čas prodloužením automatických pracovních bloků. Podle dostupného prostoru může posunout začátek pracovního bloku směrem dříve nebo prodloužit jeho konec směrem později. Taková úprava se provede pouze tehdy, pokud tím nevznikne kolize s výukou, obědem, nepřítomností nebo jinou existující událostí. Rozvrh převzatý ze systému STAG se tím nemění, protože algoritmus doplňuje pouze volný prostor kolem již uložených záznamů.

Úpravy jsou v aktuální implementaci prováděny v pětiminutových krocích. Tato hodnota slouží jako technický krok algoritmu, aby změny nevznikaly po jednotlivých minutách a zůstaly přehledné. V případě potřeby by bylo možné tuto hodnotu v implementaci změnit, protože jde o vnitřní parametr výpočtu, nikoli o údaj zadávaný uživatelem v rozhraní.

Před samotným vyrovnáváním aplikace zkontroluje dříve provedené pomocné úpravy. Pokud již neodpovídají aktuálním událostem, jsou odstraněny nebo přepočítány. To je důležité zejména při opakovaném spuštění algoritmu po ruční změně nebo po novém načtení rozvrhu. Aplikace tak nevychází z dříve upraveného stavu, ale znovu vyhodnotí skutečně uložené události.

Samostatnou částí algoritmu je kontrola obědové přestávky. Po změně začátku nebo konce pracovního dne je nutné ověřit, zda oběd stále leží uvnitř pracovního okna a zda se nepřekrývá s jinou událostí. Počet obědových přestávek se určuje podle délky pracovního okna. Pokud pracovní doba nepřesáhne čtyři hodiny, oběd se automaticky nevytváří. Pokud pracovní doba přesáhne čtyři hodiny, vytvoří se jedna obědová přestávka. Pokud pracovní doba přesáhne dvanáct hodin, vytvoří se dvě obědové přestávky.

Po vložení nebo úpravě oběda aplikace rozdělí automatické pracovní bloky tak, aby se s obědem nepřekrývaly. Pokud je oběd umístěn mimo pracovní dobu nebo jej není možné bezpečně vložit, aplikace hledá vhodnější místo v rámci pracovního dne. Cílem této kontroly je, aby výsledné události byly časově správně uspořádány a aby mezi prací a přestávkou nevznikaly překryvy.

Vyrovnávání se může opakovat vícekrát. Po každém průchodu aplikace porovná, zda se stav evidence ještě mění. Pokud další průchod již nepřinese novou změnu, zpracování se ukončí. Výsledkem je evidence pracovní doby, ve které zůstávají zachovány ručně zadané a rozvrhové události. Automatické pracovní bloky jsou upraveny podle dostupného času v jednotlivých dnech, požadovaného počtu pracovních hodin, nastaveného začátku a konce pracovního dne a pravidel pro obědovou přestávku.

Celkový průběh vyrovnávání je znázorněn v přílohách *J.1 a J.2*. Týdenní vyrovnání pracovní doby je uvedeno v přílohách *K.1 a K.2*. Kontrola a vytváření obědových přestávek je uvedena v přílohách *L.1 až L.3*.

2.6.2 Algoritmus načtení rozvrhu ze systému STAG

Algoritmus načtení rozvrhu ze systému STAG slouží k převzetí rozvrhových údajů vyučujícího do aplikace. Díky tomu uživatel nemusí jednotlivé záznamy výuky zadávat ručně. Rozvrhovou událostí se v této části rozumí časově vymezený záznam výuky převzatý ze souboru vytvořeného systémem STAG, například přednáška, cvičení nebo jiná výuková činnost. Vstupem algoritmu je soubor ve formátu CSV a výsledkem jsou rozvrhové události uložené v databázi aplikace.

Nejprve uživatel vybere soubor s rozvrhem. Aplikace se poté pokusí určit jeho kódování, protože soubor může být uložen různým způsobem podle prostředí, ve kterém byl vytvořen. Po rozpoznání kódování se načtou jednotlivé řádky souboru. Pokud je soubor prázdný nebo jej nelze správně přečíst, další zpracování se neprovede a uživatel je informován o chybě.

Po načtení souboru aplikace ověří, zda má soubor očekávanou strukturu. Kontroluje se zejména hlavička a minimální počet sloupců, ze kterých lze sestavit rozvrhovou událost. Pro vytvoření události jsou potřebné hlavně údaje o názvu výuky, popisu, začátku a konci výuky, období platnosti, rozsahu týdnů, paritě týdne a datu, podle kterého se určuje den v týdnu. Pokud tyto údaje v souboru chybí nebo soubor nemá očekávaný tvar, aplikace v načtení nepokračuje.

Poté aplikace prochází jednotlivé řádky souboru. Každý řádek rozdělí na sloupce a při tom zohlední uvozovky, aby se správně zpracoval i text obsahující oddělovač. Pokud je řádek poškozený nebo obsahuje příliš málo sloupců, aplikace ho přeskočí a запиše informaci o chybě do zprávy o průběhu načtení. Chybný řádek tedy nezastaví celé načtení, pokud lze ostatní řádky zpracovat.

U každého platného řádku aplikace ověří hodnoty potřebné pro vytvoření události. Kontroluje se zejména datum začátku a konce období, čas začátku a konce výuky, rozsah týdnů a další údaje převzaté z rozvrhu. Řádek se nepoužije například tehdy, pokud obsahuje neplatné datum, chybně zadaný čas nebo konec události není později než její začátek. Tím se zabrání vytvoření záznamů s neplatným časovým intervalem.

Z jednoho platného řádku může vzniknout více rozvrhových událostí. Jeden řádek totiž obvykle nepopisuje pouze jeden konkrétní den, ale výuku opakující se v určitém období. Aplikace proto projde celé období platnosti a vybere pouze data, která odpovídají dni v týdnu, rozsahu týdnů a

paritě týdne. Parita týdne určuje, zda se výuka koná každý týden, pouze v sudých týdnech, pouze v lichých týdnech nebo podle jiného pravidla uvedeného v rozvrhu. Každý takto nalezený termín se uloží jako samostatná rozvrhová událost.

Nově vytvořené rozvrhové události se nepovažují za automaticky vytvořené pracovní bloky. Jejich zdrojem je rozvrh vyučujícího, a proto se při dalším zpracování chovají jako pevné záznamy. Aplikace je při vyrovnávání pracovní doby nepřepisuje, ale doplňuje pracovní bloky kolem nich tak, aby nebyl narušen načtený rozvrh.

Před uložením nových rozvrhových událostí aplikace zjistí období, kterého se načtený soubor týká. V tomto období vyhledá starší rozvrhové události pocházející z předchozího načtení a označí je jako neaktivní. Nejde o okamžité fyzické odstranění z databáze, ale o vyřazení starších záznamů z dalšího používání. Díky tomu může uživatel načíst stejný rozvrh znovu a aplikace bude pracovat s aktuální verzí dat.

Současně se vytvoří záznam o dávce načtených dat. Tento záznam obsahuje identifikátor dávky, název zpracovaného souboru a čas načtení. Nově vytvořené rozvrhové události jsou s touto dávkou propojeny, takže aplikace může později určit, ze kterého souboru konkrétní záznamy pocházejí.

Po uložení rozvrhových událostí aplikace spustí navazující úpravy změněného období. Znovu vytvoří automatické pracovní bloky, zkontroluje obědové přestávky a provede vyrovnání pracovní doby. Načtení rozvrhu tedy nekončí uložením dat ze souboru. Nově převzaté rozvrhové události se tak správně promítnou do pracovního dne a automaticky vytvořené pracovní bloky se s nimi nepřekrývají.

Výsledkem tohoto postupu jsou rozvrhové události uložené v databázi, zpráva o zpracovaných a přeskočených řádcích a aktualizovaná evidence pracovní doby pro dotčené období. Průběh načtení rozvrhu ze systému STAG a následné zpracování převzatých dat jsou znázorněny v přílohách *M* a *N*.

2.7 Testování aplikace

Tato kapitola popisuje ověření funkčnosti vytvořené aplikace. Testování bylo zaměřeno na to, zda aplikace podporuje celý pracovní postup od načtení rozvrhu ze souboru, přes vytvoření a úpravu událostí, až po vyrovnání pracovní doby a vytvoření výsledného dokumentu ve formátu PDF.

Ověřována nebyla pouze samostatná funkčnost jednotlivých částí, ale především jejich návaznost. V aplikaci totiž výsledek jedné operace často ovlivňuje další kroky. Například načtení rozvrhu změni události v kalendáři, na ně navazuje vytvoření pracovních bloků, kontrola obědových přestávek a následné vyrovnaní pracovní doby.

2.7.1 Cíl a kritéria přijetí

Cílem testování bylo ověřit, že aplikace dokáže spolehlivě zpracovat běžné i chybové situace, které mohou nastat při práci s evidencí pracovní doby. Za správné chování bylo považováno, pokud aplikace načte platný soubor CSV, vytvořila odpovídající rozvrhové události, provedla navazující zpracování a umožnila vytvořit přehledný výstupní dokument ve formátu PDF.

Důležitým kritériem byla také správnost výpočtů. Aplikace musela správně započítat pracovní bloky a rozvrhové události, nezapočítávat obědovou přestávku jako práci, zohlednit neúplné týdny na začátku a konci měsíce a správně pracovat se státními svátky. Současně bylo ověřováno, že ručně zadané nebo z rozvrhu převzaté události nejsou při vyrovnavání pracovní doby nechtěně změněny.

2.7.2 Testovací prostředí

Aplikace byla testována na operačních systémech Windows 11 a Ubuntu 22.04 LTS. V obou případech byla použita platforma .NET 8, uživatelské rozhraní vytvořené pomocí frameworku Avalonia UI a lokální databáze SQLite.

Na systému Windows probíhalo testování v prostředí Windows 11 verze 23H2. Na systému Linux bylo testování provedeno v prostředí Ubuntu 22.04 LTS s běžnou konfigurací desktopového prostředí. V obou prostředích byla aplikace spuštěna nad reálnou lokální databází a byly použity vstupní soubory odpovídající běžnému použití aplikace.

2.7.3 Způsob ověřování

Testování probíhalo pomocí předem připravených ověřovacích postupů a manuální kontroly výsledků. U každého postupu byl nejprve připraven vstupní soubor nebo určitý stav aplikace, potom byla v aplikaci provedena ověřovaná činnost, a nakonec byl dosažený výsledek porovnán s očekávaným stavem.

Tento způsob byl zvolen proto, že aplikace pracuje s několika navazujícími kroky. Nestačilo tedy ověřit pouze to, zda lze otevřít soubor nebo vytvořit událost. Bylo nutné sledovat, zda po načtení rozvrhu vzniknou správné události, zda se správně doplní pracovní bloky, zda se obědová přestávka nachází ve vhodném čase a zda výsledný dokument odpovídá údajům zobrazeným v aplikaci.

Manuální kontrola byla využita zejména při ověřování uživatelského rozhraní a výsledných dokumentů. Bylo sledováno, zda jsou ovládací prvky čitelné, zda se události zobrazují ve správných pohledech, zda aplikace reaguje očekávaným způsobem při vytváření, úpravě a odstranění událostí a zda je výstupní dokument obsahově správný a přehledný.

2.7.4 Rozdíly mezi platformami

Při testování bylo zohledněno, že mezi systémy Windows a Linux mohou vznikat drobné vizuální rozdíly. Ty se mohou týkat například systémových fontů, šířky některých ovládacích prvků, vzhledu dialogových oken nebo drobných rozdílů ve vykreslování uživatelského rozhraní.

Za přijatelný stav bylo považováno, pokud tyto rozdíly neovlivnily použitelnost aplikace ani správnost výstupních dat. Text musel být na obou platformách čitelný, ovládací prvky musely být dostupné a nesmělo docházet k překrývání obsahu. U dokumentů ve formátu PDF byla posuzována především obsahová správnost, nikoli binární shoda souborů.

2.7.5 Ověřované oblasti testování

Testování bylo rozděleno do několika oblastí podle hlavních funkcí aplikace. První oblast se zaměřila na načtení rozvrhu ze systému STAG. Bylo ověřováno, zda aplikace správně zpracuje platný soubor CSV, vytvoří odpovídající rozvrhové události a při opakovaném načtení stejného období pracuje s aktuální verzí dat.

Druhá oblast se týkala chybových vstupů. Testovány byly například soubory s neplatným datem, chybným časem, chybějícími sloupci nebo poškozenými řádky. Cílem bylo ověřit, že aplikace na tyto situace reaguje chybovým hlášením nebo problematický řádek bezpečně přeskočí, aniž by tím poškodila již uložená data.

Třetí oblast byla zaměřena na pravidla evidence pracovní doby. Ověřovaly se neúplné týdny, státní svátky, pracovní dny přesahující hranici měsíce, kombinace výuky a služební cesty v jednom dni a krátkodobé nepřítomnosti. V těchto případech bylo důležité zkontrolovat, zda aplikace správně počítá odpracovaný a neodpracovaný čas.

Čtvrtá oblast se věnovala ručním úpravám uživatele. Testováno bylo vytvoření nové události, změna existující události, odstranění záznamu, úprava začátku nebo konce pracovního dne a změna obědové přestávky. Ověřovalo se, zda aplikace po těchto zásazích správně přepočítá navazující pracovní bloky a zároveň zachová ručně zadané údaje.

Poslední oblast se zaměřila na výsledný dokument a použitelnost aplikace. Byla kontrolována správnost dokumentu PDF, přehlednost tabulky, součty pracovních hodin, přesčasy, neodpracovaná doba a chování aplikace při delším zpracování. Součástí ověření byla také pravidla pro automatické vytváření obědových přestávek, tedy vytvoření jedné přestávky u pracovních dnů delších než čtyři hodiny a vytvoření dvou přestávek u pracovních dnů delších než dvanáct hodin.

2.7.6 Souhrn ověřených scénářů

Podrobný přehled ověřených scénářů je uveden v *příloze O*. Tabulka v této příloze shrnuje hlavní testované oblasti, konkrétní ověřované situace a způsob, jakým aplikace daný stav řeší. V hlavním textu práce jsou proto popsány pouze skupiny testů a jejich význam, zatímco detailní přehled je přesunut do přílohy.

2.7.7 Shrnutí testování a známá omezení

Provedené testování ukázalo, že aplikace zvládá hlavní pracovní postupy, pro které byla navržena. Bylo ověřeno načtení rozvrhu ze systému STAG, zpracování chybových vstupů, opakované načtení stejného období, vytváření a úprava událostí, automatické doplňování obědových přestávek, ukládání údajů do lokální databáze SQLite a tvorba výsledného dokumentu PDF.

Testování zároveň potvrdilo, že aplikace zachovává konzistentní stav dat i v situacích, kdy vstupní soubor obsahuje neplatné nebo neúplné údaje. Chybné řádky nejsou použity pro vytvoření událostí a aplikace uživatele informuje o problému. Tím se omezuje riziko vytvoření nesprávných záznamů v databázi.

Za známé omezení lze považovat především drobné vizuální rozdíly mezi platformami Windows a Linux. Tyto rozdíly se mohou projevit ve vzhledu ovládacích prvků nebo v detailech vykreslení textu, neměly by však ovlivnit obsahovou správnost evidence ani použitelnost aplikace. Dalším omezením je delší doba zpracování u rozsáhlejších dat, například při opakovaném načtení rozvrhu za delší období.

Z výsledků testování vyplývá, že aplikace splňuje stanovené požadavky a je použitelná pro praktickou práci s evidencí pracovní doby. Testy potvrdily správnost hlavních algoritmů, konzistenci uložených dat a správnost výsledných výstupů.

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout, implementovat a ověřit aplikaci určenou pro evidenci pracovní doby. Navržené řešení bylo vytvořeno s důrazem na přehlednost, srozumitelnost, spolehlivost a omezení chybovosti při zpracování údajů. Výsledkem práce je desktopová aplikace, která umožňuje evidenci a úpravu záznamů pracovní doby, import vstupních dat a generování výsledných výstupů ve formátu PDF.

V teoretické části byly vymezeny základní principy evidence pracovní doby, analyzovány současné přístupy k jejímu vedení a formulovány požadavky na vhodné softwarové řešení. Následně byly popsány architektonické a technologické prostředky použité při realizaci aplikace. Tato část vytvořila teoretický základ pro návrh praktického řešení.

Praktická část práce se zaměřila na návrh, implementaci a ověření vytvořené aplikace. Byla popsána struktura systému, databázový model, uživatelské rozhraní i algoritmy, které zajišťují klíčové operace aplikace. Zvláštní pozornost byla věnována zejména algoritmu vyrovnávání hodin a algoritmu importu rozvrhu ze systému STAG, protože právě tyto části představují základní logiku celého řešení.

Součástí práce bylo rovněž testování aplikace. Testovací scénáře byly zaměřeny na běžné i hraniční situace, včetně importu dat, vyrovnávání pracovní doby, generování PDF výstupů a práce s uživatelským rozhraním. Výsledky testování ukázaly, že aplikace splňuje stanovené požadavky, správně zpracovává data a poskytuje stabilní a přehledné výstupy. Současně bylo ověřeno, že aplikace vykazuje srovnatelné chování na platformách Windows i Linux.

Přínosem práce je vytvoření funkčního softwarového řešení, které může přispět ke zjednodušení evidence pracovní doby a ke zefektivnění její administrativní části. Do budoucna by bylo možné aplikaci dále rozšířit například o export do dalších formátů, podrobnější nastavení pracovních režimů nebo napojení na další informační systémy.

POUŽITÁ LITERATURA

1. ČESKO, 2006. Zákon č. 262/2006 Sb., zákoník práce. In: *Zákony pro lidi [online]*. [cit. 2026-04-26]. Dostupné z: <https://www.zakonyprolidi.cz/cs/2006-262>
2. MINISTERSTVO PRÁCE A SOCIÁLNÍCH VĚCÍ, 2026. VIII.9 Evidence pracovní doby. In: *Příručka pro personální agendu a odměňování zaměstnanců [online]*. [cit. 2026-04-26]. Dostupné z: <https://ppropo.mpsv.cz/VIII9Evidencepracovnidoby>
3. STÁTNÍ ÚŘAD INSPEKCE PRÁCE, bez data. Rozvrhování pracovní doby. In: *suip.gov.cz [online]*. [cit. 2026-04-26]. Dostupné z: https://suip.gov.cz/vsechny-clanky/-/asset_publisher/BwIpyjT9IXe0/content/rozvrhovani-pracovni-do-1
4. ČESKO, 1998. Zákon č. 111/1998 Sb., o vysokých školách. In: *Zákony pro lidi [online]*. [cit. 2026-04-26]. Dostupné z: <https://www.zakonyprolidi.cz/cs/1998-111>
5. SOUDNÍ DVŮR EVROPSKÉ UNIE, 2019. Member States must require employers to set up a system enabling the duration of daily working time to be measured. In: *curia.europa.eu [online]*. 14. 5. 2019 [cit. 2026-04-26]. Dostupné z: <https://curia.europa.eu/jcms/upload/docs/application/pdf/2019-05/cp190061en.pdf>
6. EUROFOUND, 2020. Regulations to address work–life balance in digital flexible working arrangements. In: *eurofound.europa.eu [online]*. [cit. 2026-04-26]. Dostupné z: <https://www.eurofound.europa.eu/en/publications/all/regulations-address-work-life-balance-digital-flexible-working-arrangements>
7. EUROSTAT, bez data. Job autonomy and pressure at work – statistics. In: *Statistics Explained [online]*. [cit. 2026-04-26]. Dostupné z: https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Job_autonomy_and_pressure_at_work_-_statistics
8. IBM, bez data. What Is Requirements Management? In: *ibm.com [online]*. [cit. 2026-04-26]. Dostupné z: <https://www.ibm.com/think/topics/what-is-requirements-management>
9. ISO 25000, bez data. ISO/IEC 25010. In: *iso25000.com [online]*. [cit. 2026-04-26]. Dostupné z: <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>
10. CMU/SEI, 2010. What Is Your Definition of Software Architecture? In: *sei.cmu.edu [online]*. Carnegie Mellon University, 2010 [cit. 2026-04-26]. Dostupné z: <https://www.sei.cmu.edu/library/what-is-your-definition-of-software-architecture/>
11. AZURE ARCHITECTURE CENTER, 2025. Architecture styles – přehled. In: *learn.microsoft.com [online]*. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/cs-cz/azure/architecture/guide/architecture-styles/>

12. AVALONIA UI, 2025. *The MVVM Pattern*. In: docs.avaloniaui.net [online]. Avalonia, 2025 [cit. 2026-04-26]. Dostupné z: <https://docs.avaloniaui.net/docs/concepts/the-mvvm-pattern/>
13. REACTIVEUI COMMUNITY, 2025. *The Zen of the ViewModel*. In: reactiveui.net [online]. ReactiveUI, 2025 [cit. 2026-04-26]. Dostupné z: <https://www.reactiveui.net/docs/handbook/view-models/>
14. MICROSOFT, 2023. *ASP.NET MVC Overview – Model–View–Controller*. In: learn.microsoft.com [online]. Microsoft, 2023 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/overview/aspnet-mvc-overview>
15. MICROSOFT, 2023. *Common web application architectures – monolitická aplikace*. In: learn.microsoft.com [online]. Microsoft, 2023 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
16. FOWLER, Martin, 2015. *Monolith First*. In: martinofowler.com [online]. 2015 [cit. 2026-04-26]. Dostupné z: <https://martinofowler.com/bliki/MonolithFirst.html>
17. MICROSOFT, 2025. *Microservices architecture style – přehled a trade-offy*. In: learn.microsoft.com [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>
18. MICROSOFT, 2025. *.NET – Build modern apps and services*. In: dotnet.microsoft.com [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://dotnet.microsoft.com/en-us/>
19. MICROSOFT, 2025. *A tour of the C# language*. In: learn.microsoft.com [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/csharp/tour-of-csharp/overview>
20. MICROSOFT, 2024. *Entity Framework Core – SQLite provider*. In: learn.microsoft.com [online]. Microsoft, 2024 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/cs-cz/ef/core/providers/sqlite/>
21. SQLITE, 2024. *SQL As Understood By SQLite*. In: sqlite.org [online]. SQLite, 2024 [cit. 2026-04-26]. Dostupné z: <https://www.sqlite.org/lang.html>
22. REACTIVEUI COMMUNITY, 2025. *Getting Started*. In: reactiveui.net [online]. ReactiveUI, 2025 [cit. 2026-04-26]. Dostupné z: <https://www.reactiveui.net/docs/getting-started/>

23. AVALONIA COMMUNITY, 2024. *Behaviors v Avalonia UI*. In: *docs.avaloniaui.net* [online]. Avalonia Community, 2024 [cit. 2026-04-26]. Dostupné z: <https://docs.avaloniaui.net/docs/controls/behaviors>
24. QUESTPDF, 2025. *QuestPDF – C# PDF Generation Library*. In: *questpdf.com* [online]. QuestPDF, 2025 [cit. 2026-04-26]. Dostupné z: <https://www.questpdf.com/>
25. ARTIFEX SOFTWARE, 2025. *Ghostscript – official documentation*. In: *ghostscript.com* [online]. Artifex Software, 2025 [cit. 2026-04-26]. Dostupné z: <https://www.ghostscript.com/>
26. HABJAN, John a kol., 2025. *Ghostscript.NET – .NET wrapper pro Ghostscript*. In: *github.com/jhabjan/Ghostscript.NET* [online]. 2025 [cit. 2026-04-26]. Dostupné z: <https://github.com/jhabjan/Ghostscript.NET>
27. MICROSOFT, 2025. *Visual Studio – Product documentation*. In: *learn.microsoft.com* [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/ide/?view=vs-2022>
28. SQLITESTUDIO, 2025. *SQLiteStudio – Official website*. In: *sqlitestudio.pl* [online]. 2025 [cit. 2026-04-26]. Dostupné z: <https://sqlitestudio.pl/>

SEZNAM PŘÍLOH

Příloha A: Zdrojový kód aplikace TeacherScheduleApp.

Příloha B: Adresářová struktura projektu TeacherScheduleApp.

Příloha C: Levý panel hlavní stránky aplikace.

Příloha D: Pravý panel hlavní stránky aplikace.

Příloha E: Denní pohled aplikace.

Příloha F: Týdenní pohled aplikace.

Příloha G: Měsíční pohled aplikace.

Příloha H: Globální nastavení aplikace.

Příloha I: Ukázka výstupu evidence pracovní doby ve formátu PDF.

Příloha J: Celkový průběh algoritmu vyrovnávání hodin.

Příloha J.1: Určení rozsahu a příprava vyrovnání.

Příloha J.2: Zpracování týdnů a dokončení vyrovnání.

Příloha K: Zjednodušené schéma týdenního vyrovnání pracovní doby.

Příloha K.1: Výpočet týdenního salda.

Příloha K.2: Úpravy týdne podle salda.

Příloha L: Kontrola a vytváření obědových přestávek.

Příloha L.1: Základní kontrola dne.

Příloha L.2: Rozhodnutí o vytvoření obědové přestávky.

Příloha L.3: Umístění a uložení obědové přestávky.

Příloha M: Základní průběh načtení rozvrhu ze systému STAG.

Příloha N: Uložení načtených dat a navazující zpracování evidence.

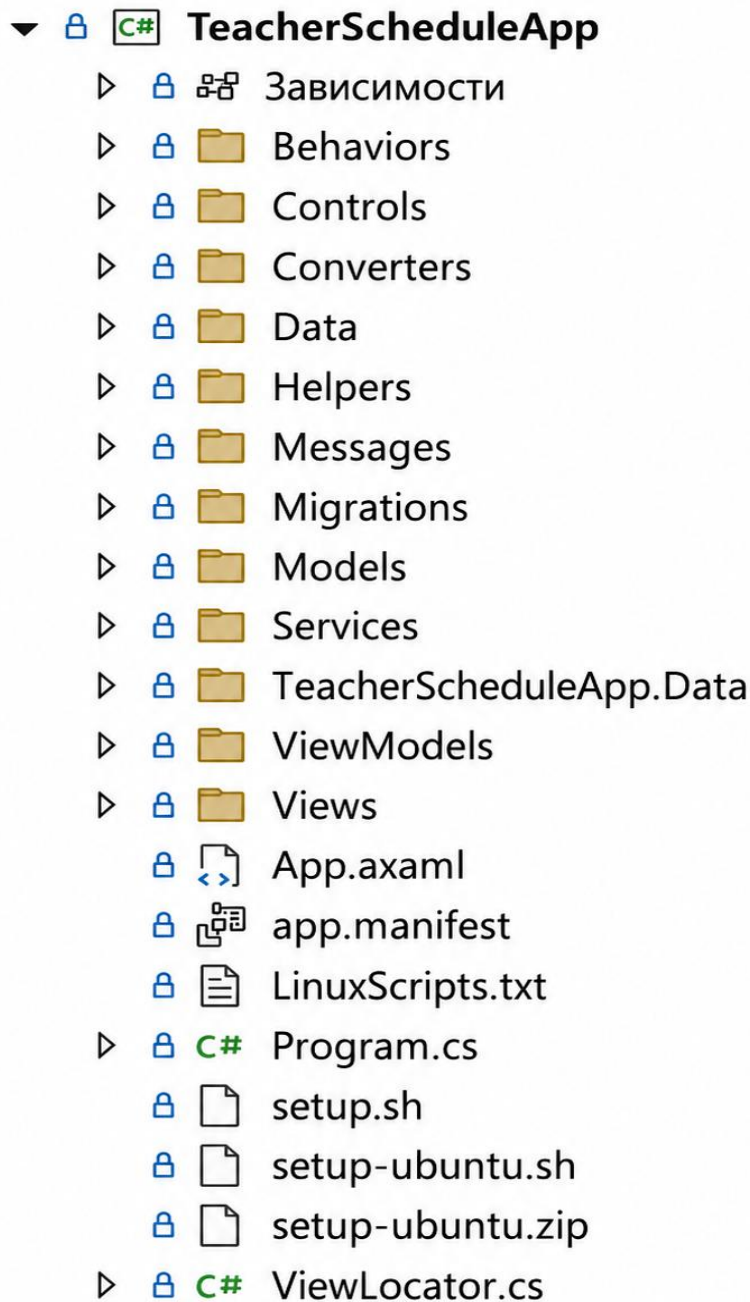
Příloha O: Souhrnná tabulka ověřených scénářů a řešených stavů.

PŘÍLOHA A: Zdrojový kód aplikace TeacherScheduleApp

Tato příloha obsahuje archiv ZIP se zdrojovým projektem aplikace TeacherScheduleApp vytvořeným ve vývojovém prostředí Visual Studio 2022.

PŘÍLOHA B: Adresářová struktura projektu TeacherScheduleApp

Tato příloha obsahuje ukázkou adresářové struktury projektu TeacherScheduleApp ve vývojovém prostředí Visual Studio. Struktura zachycuje hlavní složky a soubory projektu, například Views, ViewModels, Models, Data, Services, Helpers, Controls, Converters, Behaviors a Messages. Přehled slouží k lepší orientaci v rozdělení zdrojového kódu aplikace.



PŘÍLOHA C: Levý panel hlavní stránky aplikace

Tato příloha obsahuje ukázkou levého panelu hlavní stránky aplikace. Levý panel slouží k ovládání aplikace, výběru data, načtení rozvrhu, vytvoření nové události, otevření globálního nastavení a zobrazení náhledu evidence pracovní doby. Součástí panelu je také přehled načtených souborů a nastavení pracovního dne.

Vytvořit událost

Načíst rozvrh

Globální nastavení

EPD náhled

duben 2026

^

v

P	Ú	S	Č	P	S	N
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

Načtený soubory

^

getRozvrhByUcitel-2025-04-11-14...

11.02.2025 → 07.05.2025

Událostí: 142

Smazat

Rozvrh

v

Nastavení pro den:

14.04.2026

Příchod:

08:30

Odchod:

17:00

Oběd od:

12:30

Oběd do:

13:00

Uložit nastavení

Nastaveno pro den:

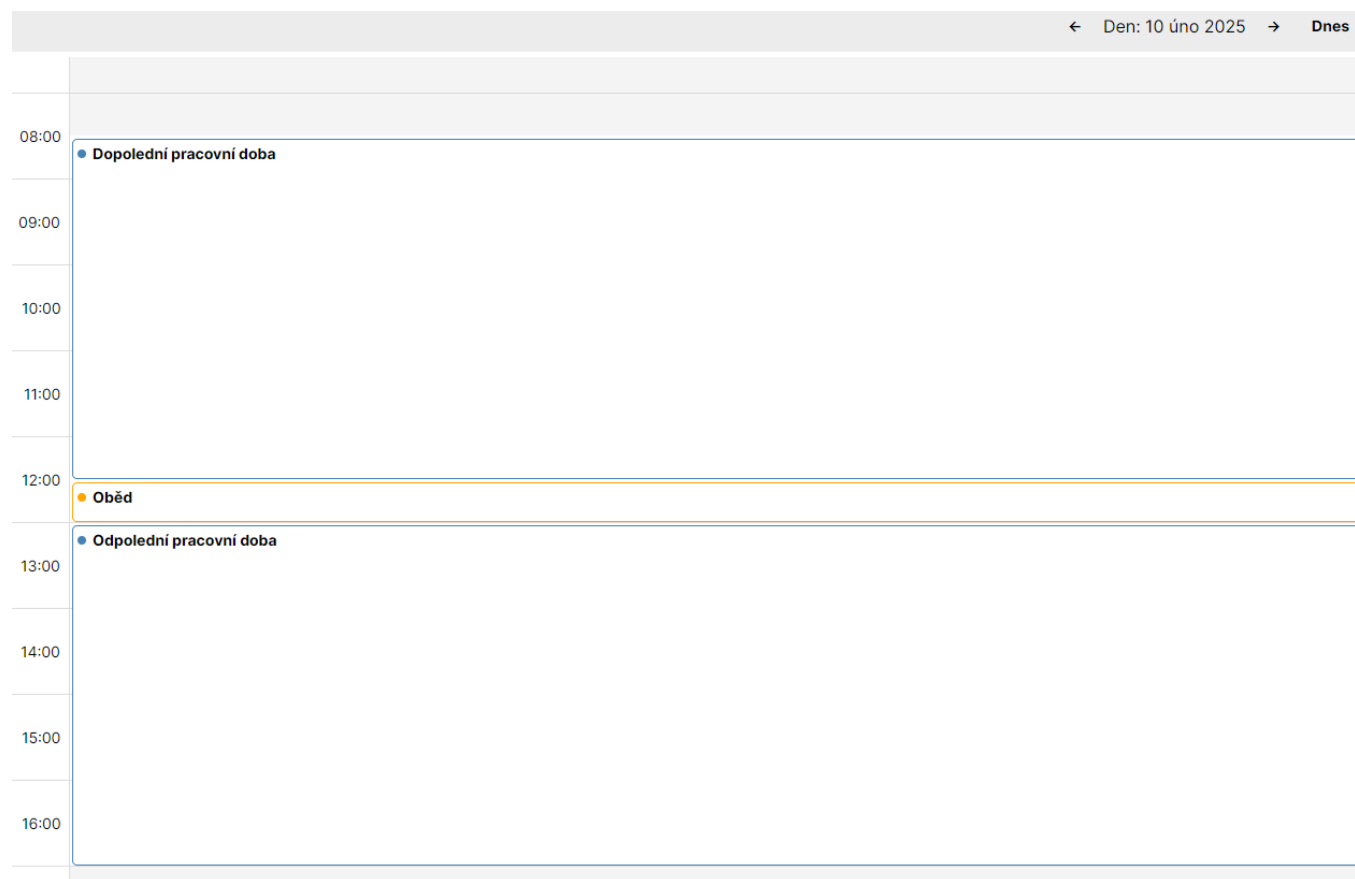
08.00

Skutečně započteno:

08:00 / 08:00

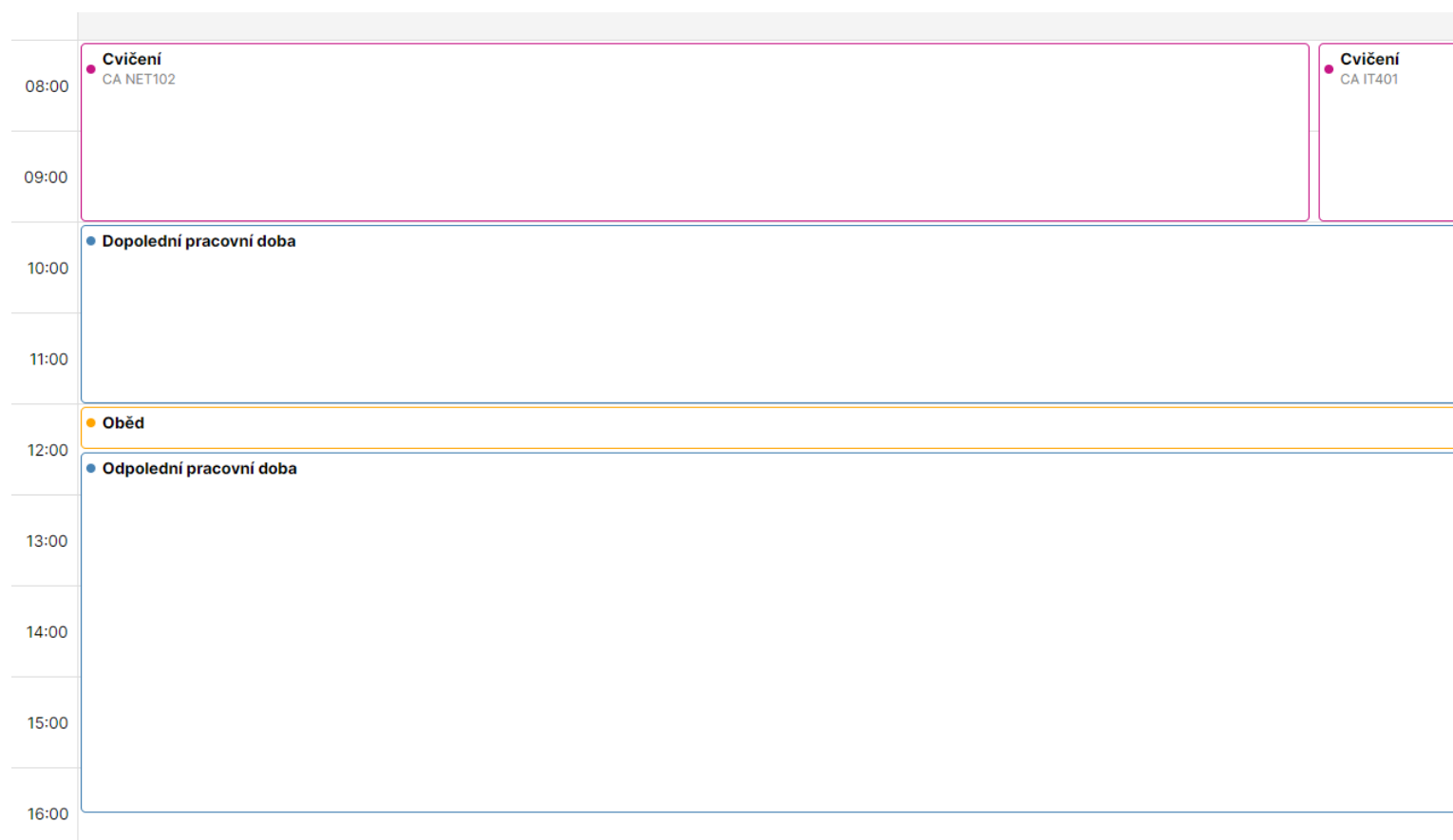
PŘÍLOHA D: Pravý panel hlavní stránky aplikace

Tato příloha obsahuje ukázkou pravé části hlavní stránky aplikace. Pravý panel slouží k zobrazení evidence pracovní doby v kalendářové podobě. Uživatel zde vidí jednotlivé události a pracovní bloky v časovém rozvržení a může kontrolovat jejich umístění v rámci vybraného dne, týdne nebo měsíce. Zobrazení je záměrně oříznuto tak, aby byly dobře čitelné hlavní prvky pravého panelu; části obrazovky mimo výřez nejsou pro účel této ukázky rozhodující.



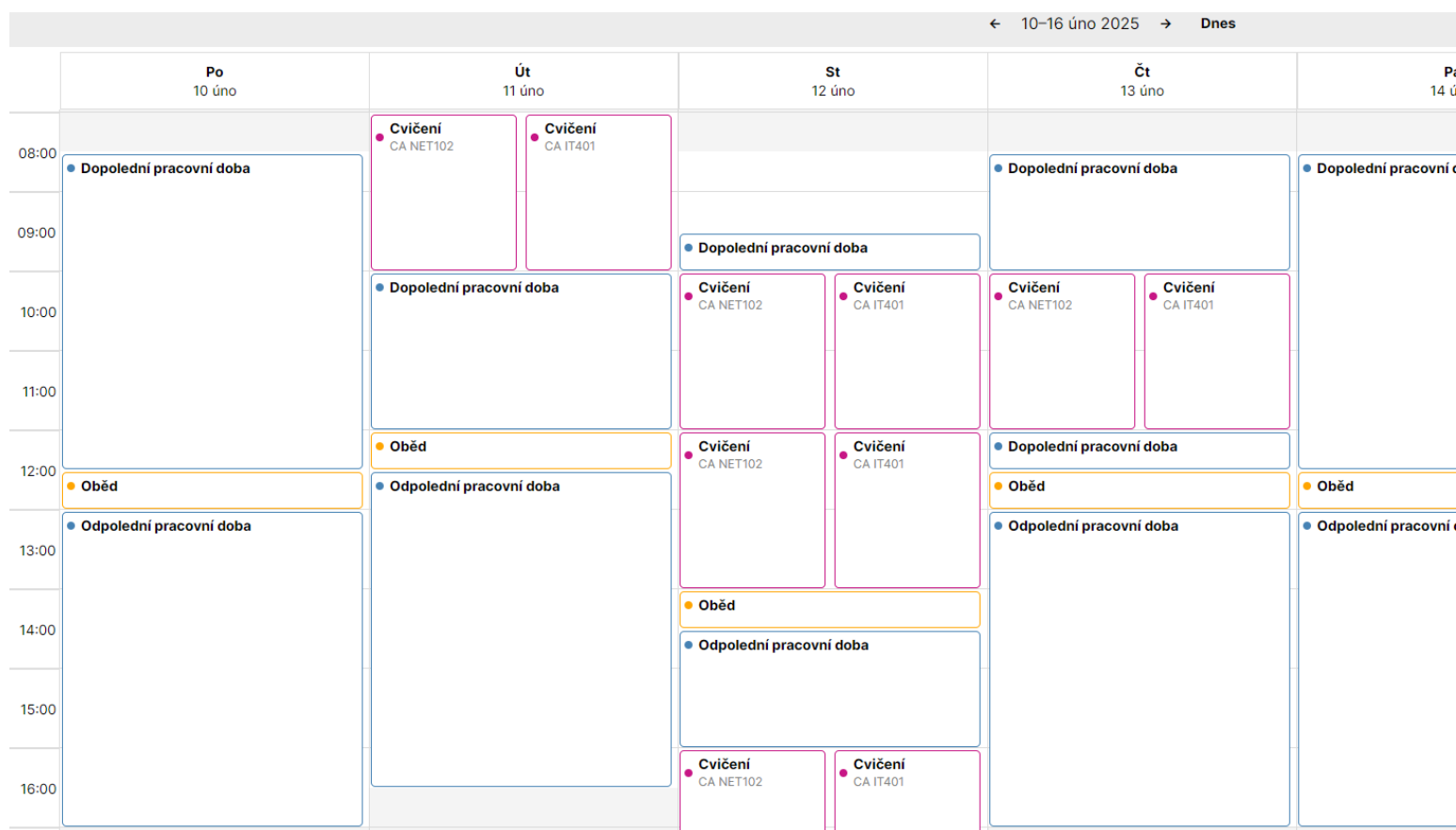
PŘÍLOHA E: Denní pohled aplikace

Tato příloha obsahuje ukázkou denního pohledu aplikace TeacherScheduleApp. Pohled slouží k detailnímu zobrazení událostí a časových bloků v rámci jednoho konkrétního dne. Zobrazení je záměrně oříznuto tak, aby byly dobře čitelné časové bloky a jejich umístění v rámci dne; části obrazovky mimo výřez nejsou pro účel této ukázky rozhodující.



PŘÍLOHA F: Týdenní pohled aplikace

Tato příloha obsahuje ukázkou týdenního pohledu aplikace. Týdenní pohled umožňuje zobrazit pracovní události v rámci několika po sobě jdoucích dnů a poskytuje přehled o rozložení pracovní doby v týdenním kontextu. Zobrazení je záměrně oříznuto tak, aby byly dobře čitelné jednotlivé dny a události v týdenním rozvržení; části obrazovky mimo výřez nejsou pro účel této ukázky rozhodující.



PŘÍLOHA G: Měsíční pohled aplikace

Tato příloha obsahuje ukázkou měsíčního pohledu aplikace. Měsíční pohled poskytuje širší kalendářový přehled o evidovaných událostech a umožňuje orientačně kontrolovat rozložení pracovní doby v rámci celého měsíce. Zobrazení je záměrně oříznuto tak, aby byly dobře čitelné jednotlivé dny a jejich základní obsah; části obrazovky mimo výřez nejsou pro účel této ukázky rozhodující.

← únor 2025 → Dnes				
Po	Út	St	Čt	Pá
27	28	29	30	
08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	
3	4	5	6	
08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	
10	11	12	13	
08:30 Dopolední pracovní doba 12:30 Oběd 13:00 Odpolední pracovní doba	08:00 Cvičení 08:00 Cvičení 10:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	09:30 Dopolední pracovní doba 10:00 Cvičení 10:00 Cvičení 12:00 Cvičení 12:00 Cvičení 14:00 Oběd	08:30 Dopolední pracovní doba 10:00 Cvičení 10:00 Cvičení 12:00 Dopolední pracovní doba 12:30 Oběd 13:00 Odpolední pracovní doba	
17	18	19	20	
08:30 Dopolední pracovní doba 12:30 Oběd	08:00 Cvičení 08:00 Cvičení	08:30 ауцкау 10:00 Cvičení	08:30 Dopolední pracovní doba 10:00 Cvičení	

PŘÍLOHA H: Globální nastavení aplikace

Tato příloha obsahuje ukázkou obrazovky globálního nastavení aplikace. Globální nastavení slouží k úpravě výchozích časových parametrů pracovního dne, přestávek a dalších údajů, které ovlivňují zpracování evidence pracovní doby a vytváření výsledných výstupů.

[← Zpět ke kalendáři](#)

Rok 2025 · Letní semestr

Rok: 2025

☐ Zimní ☒ Letní

Globální pracovní doba · Letní semestr

Od: 08:30

Do: 17:00

Pracovní doba · Letní semestr

Pondělí

Úterý

Středa

Čtvrtek

Pátek

Příchod: 08:30

Odchod: 17:00

Oběd, začátek: 12:30

Oběd, konec: 13:00

Další nastavení

Údaje pro PDF

Jméno: Radek Matoušek

Útvar: Katedra informačních technologií

Pauzy

Min. délka pauzy: 00:15

Max. délka pauzy: 01:00

Názvy událostí

Před obědem: Dopolodní pracovní doba

Pro oběd: Oběd

Po obědě: Odpolední pracovní doba

Uložit nastavení

PŘÍLOHA I: Ukázka výstupu evidence pracovní doby ve formátu PDF

Tato příloha obsahuje ukázkou výsledného PDF dokumentu evidence pracovní doby. Výstup zobrazuje měsíční přehled pracovní doby, přestávek, odpracovaného času, přesčasů, neodpracované doby a souhrnných údajů za zvolené období.

02-2025

[Uložit PDF](#)

Evidence pracovní doby, včetně přestávek v práci a práce přesčas

Fakulta elektrotechniky a informatiky
jméno: Radek Matoušek
útvár: Katedra informačních technologií

pracovní doba: 08:00–16:30 hod.
docházka za měsíc: únor 2025

Fond prac. doby:
160 hodin

Den	Začátek pracovní doby	1. přestávka		2. přestávka		Konec pracovní doby	Odpracováno	Přesčas	Neodprac.	Poznámka *)
		Od	Do	Od	Do					
1.										
2.										
3.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
4.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
5.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
6.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
7.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
8.										
9.										
10.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
11.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
12.	09:30	14:00	14:30			18:00	08:00:00	00:00:00	00:00:00	
13.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
14.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
15.										
16.										
17.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
18.	12:00					16:00	04:00:00	00:00:00	04:00:00	D
19.							00:00:00	00:00:00	08:00:00	D
20.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
21.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
22.										
23.										
24.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
25.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
26.	09:30	14:00	14:30			18:00	08:00:00	00:00:00	00:00:00	
27.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
28.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
Celkem							148:00:00	00:00:00	12:00:00	

kontr. č. Σ odprac. - přesčas + neodprac. hodin (odpovídá FPD): 160:00:00

podpis zaměstnance:

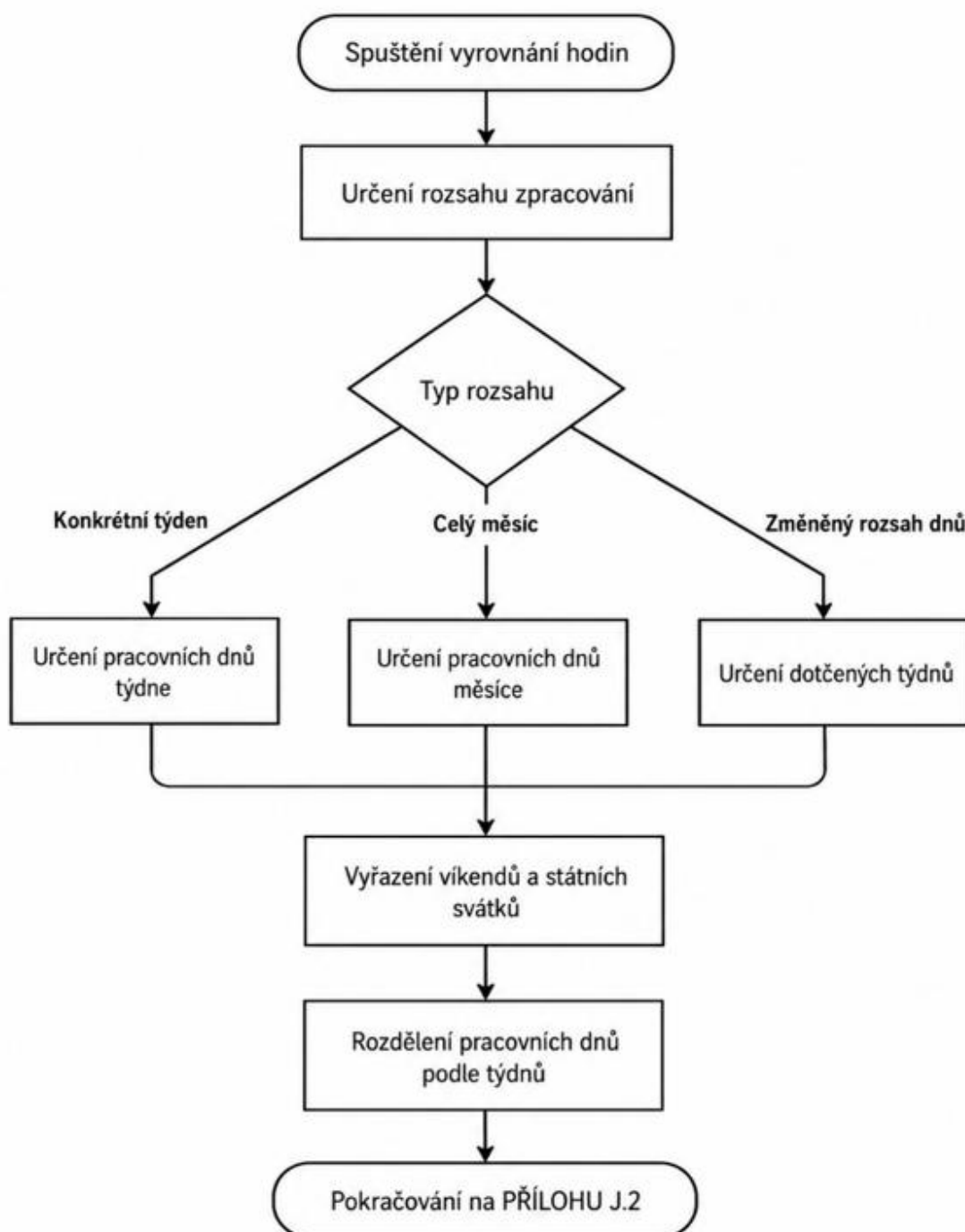
podpis nadřízeného pracovníka:

*) do poznámky: D – dovolená, N – nemoc, OČR – ošetřování, L – lékař, PC – pracovní cesta, S – svátek a ostatní dny pracovního klidu

PŘÍLOHA J: Celkový průběh algoritmu vyrovnávání hodin

Tato příloha obsahuje bloková schémata celkového průběhu algoritmu vyrovnávání hodin. Z důvodu lepší čitelnosti je schéma rozděleno do dvou částí. První část zachycuje určení rozsahu zpracování a přípravu pracovních dnů pro vyrovnání. Druhá část navazuje samotným zpracováním týdnů, kontrolou neúplných týdnů, měsíční normalizací a ukončením vyrovnávání po stabilizaci výsledného stavu evidence.

PŘÍLOHA J.1: Určení rozsahu a příprava vyrovnání



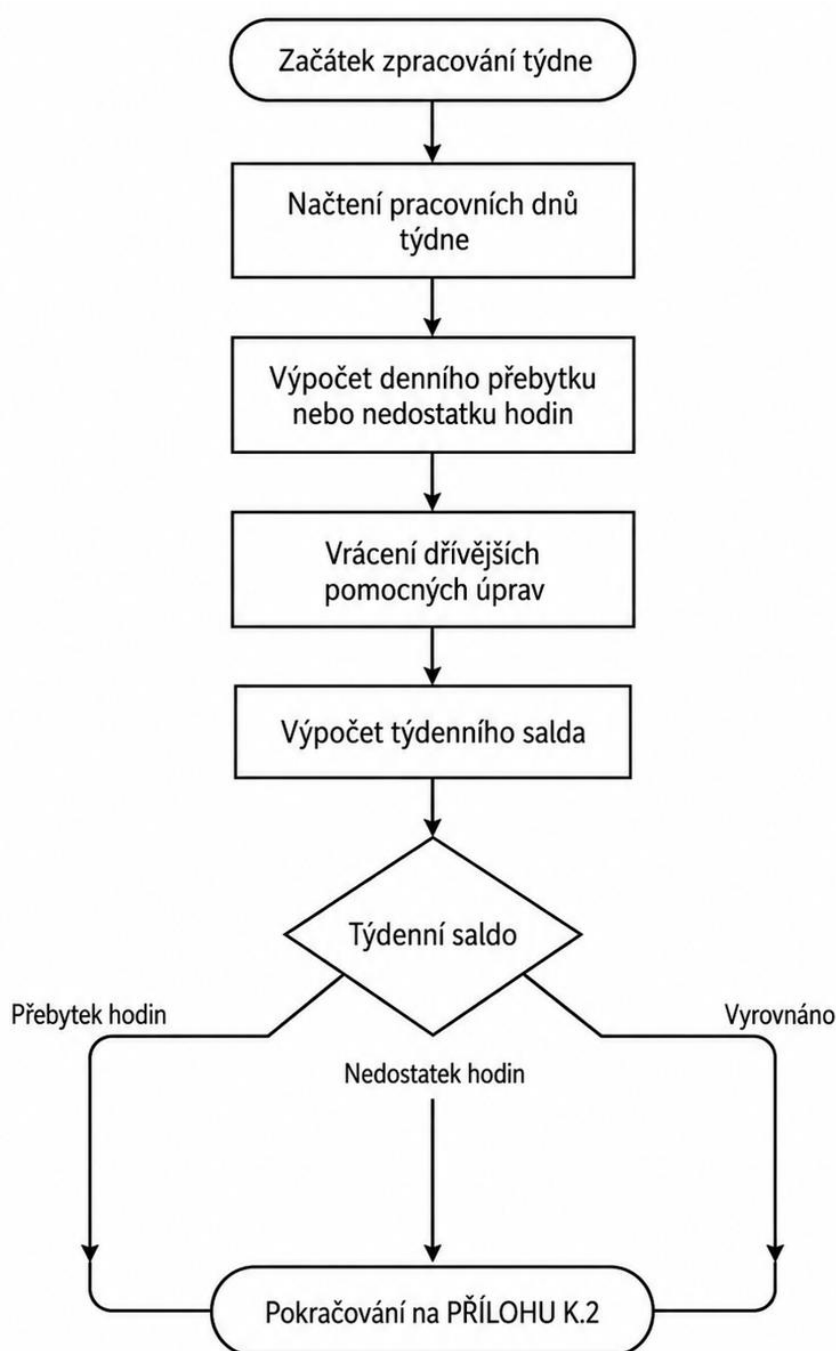
PŘÍLOHA J.2: Zpracování týdnů a dokončení vyrovnání



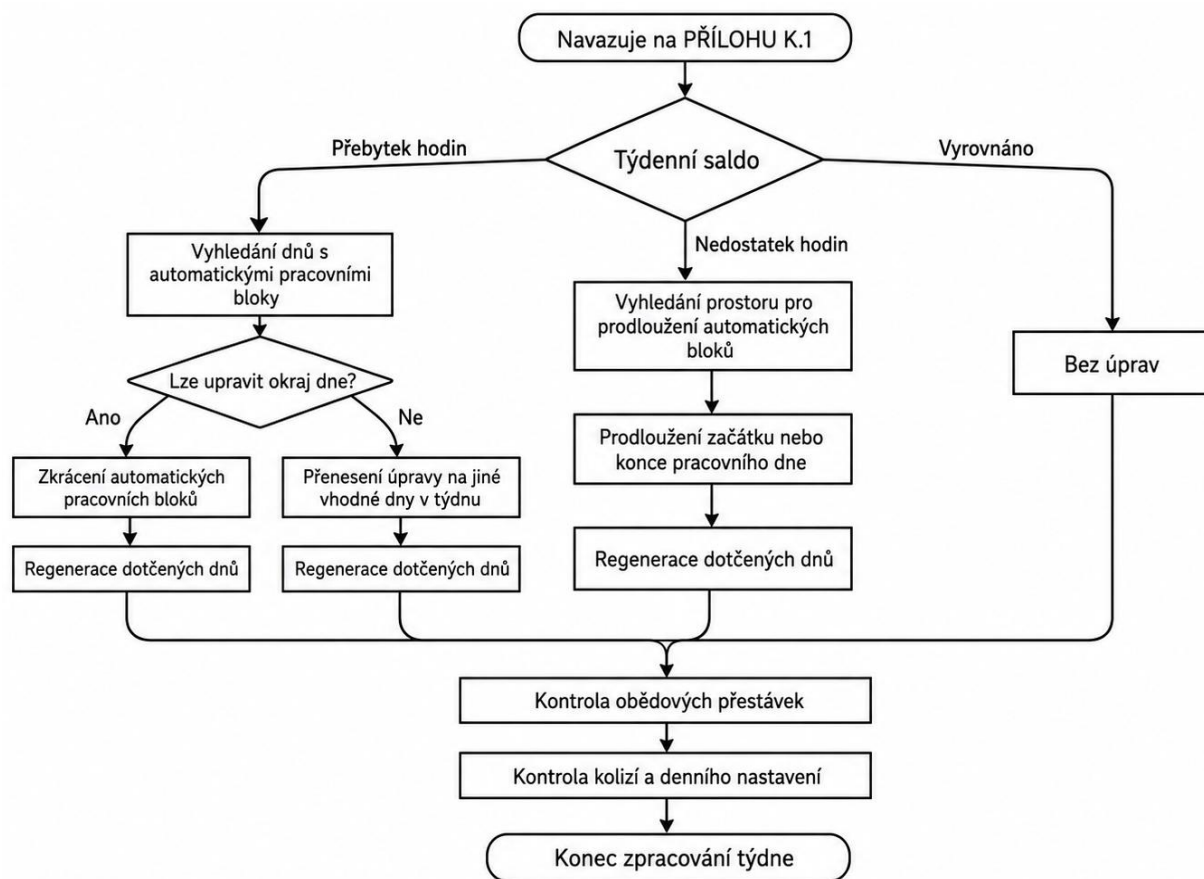
PŘÍLOHA K: Zjednodušené schéma týdenního vyrovnání pracovní doby

Tato příloha obsahuje zjednodušené blokové schéma týdenního vyrovnání pracovní doby. Z důvodu zachování čitelnosti je schéma rozděleno do dvou částí. První část popisuje načtení pracovních dnů týdne, výpočet denních rozdílů a určení týdenního salda. Druhá část znázorňuje postup při přebytku nebo nedostatku hodin, úpravy automatických pracovních bloků a následnou kontrolu obědových přestávek a denního nastavení.

PŘÍLOHA K.1: Výpočet týdenního salda



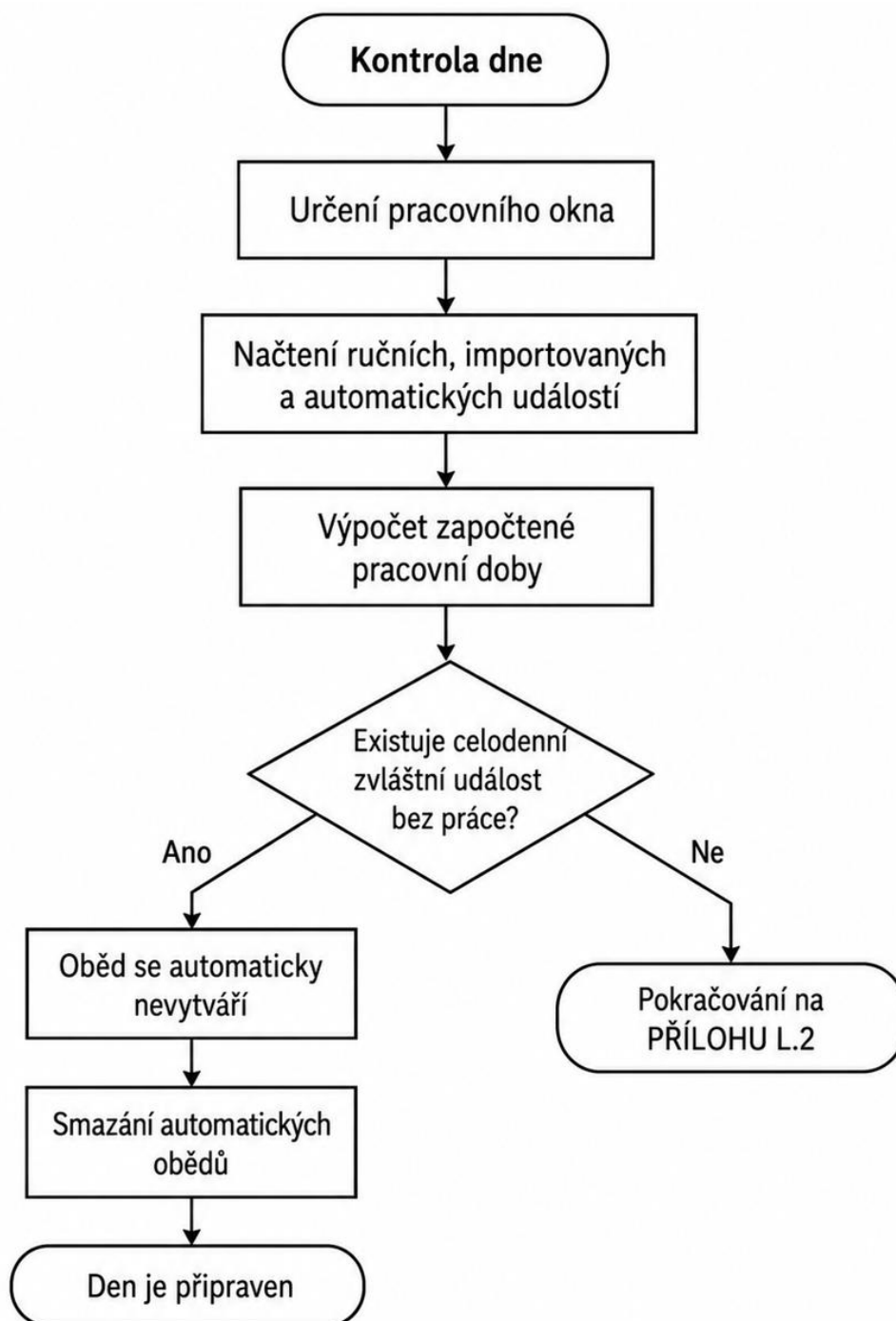
PŘÍLOHA K.2: Úpravy týdne podle salda



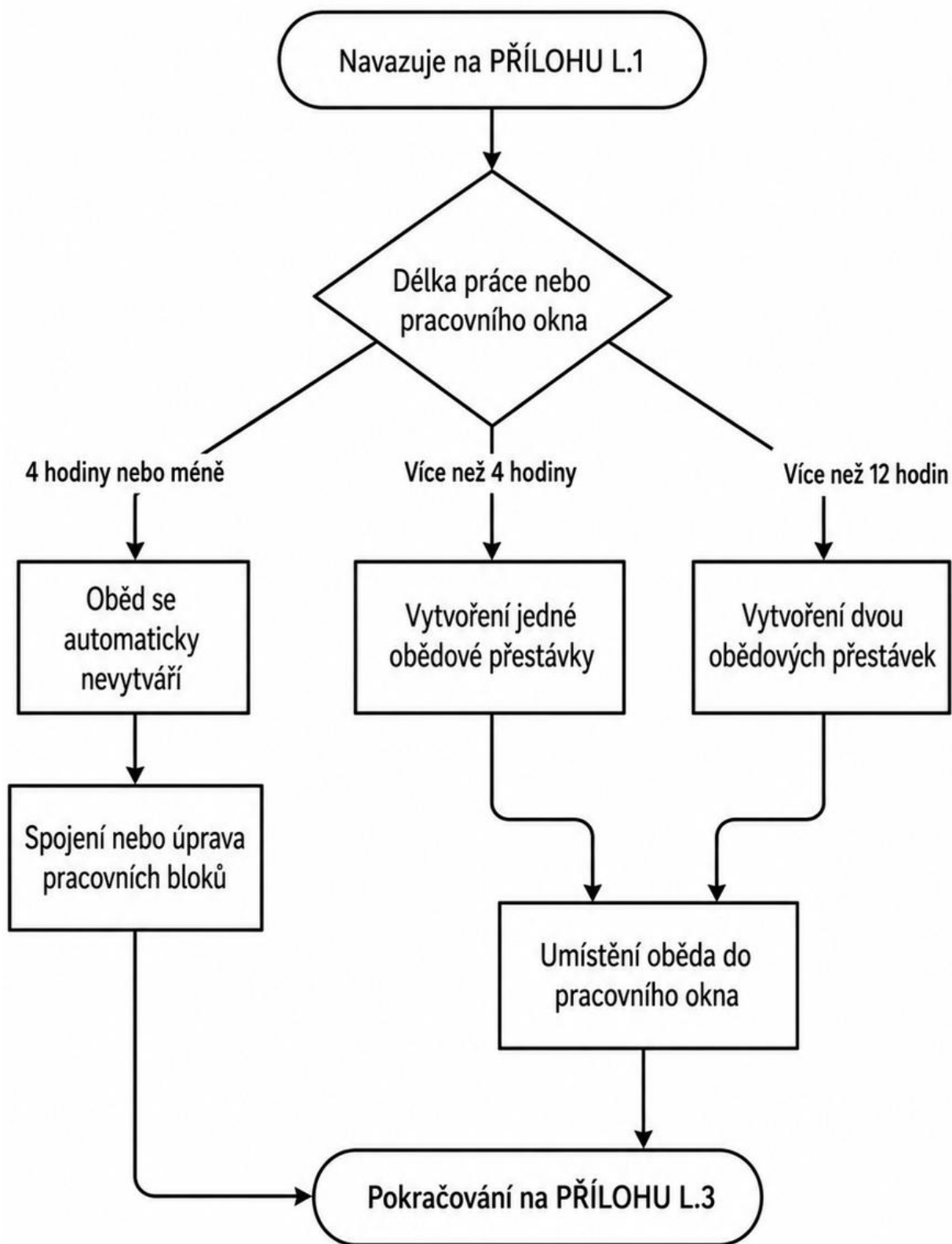
PŘÍLOHA L: Kontrola a vytváření obědových přestávek

Tato příloha obsahuje blokové schéma kontroly a vytváření obědových přestávek. Protože celý postup obsahuje více rozhodovacích kroků, je z důvodu lepší čitelnosti rozdělen do tří částí. První část zachycuje základní kontrolu dne a načtení událostí. Druhá část popisuje rozhodnutí, zda má být obědová přestávka vytvořena podle délky pracovní doby. Třetí část znázorňuje umístění oběda, řešení případné kolize a následné rozdělení pracovních bloků.

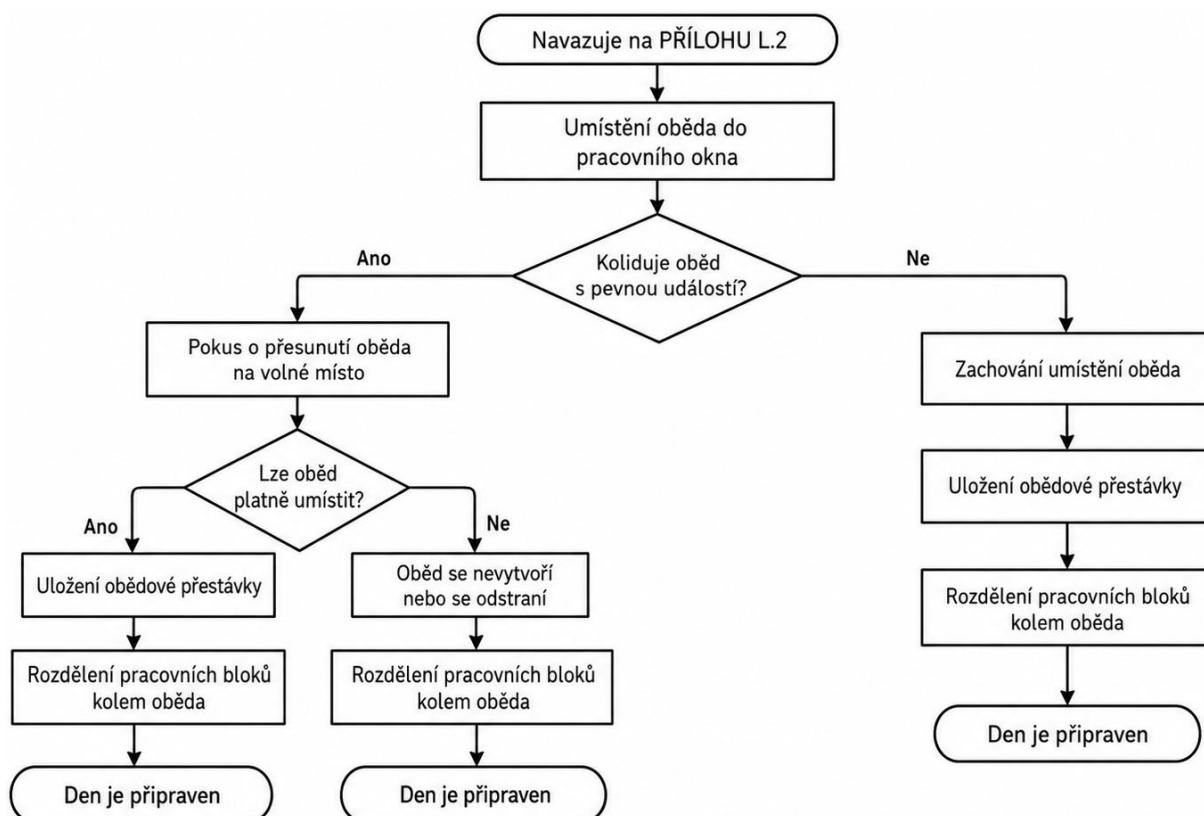
PŘÍLOHA L.1: Základní kontrola dne



PŘÍLOHA L.2: Rozhodnutí o vytvoření obědové přestávky

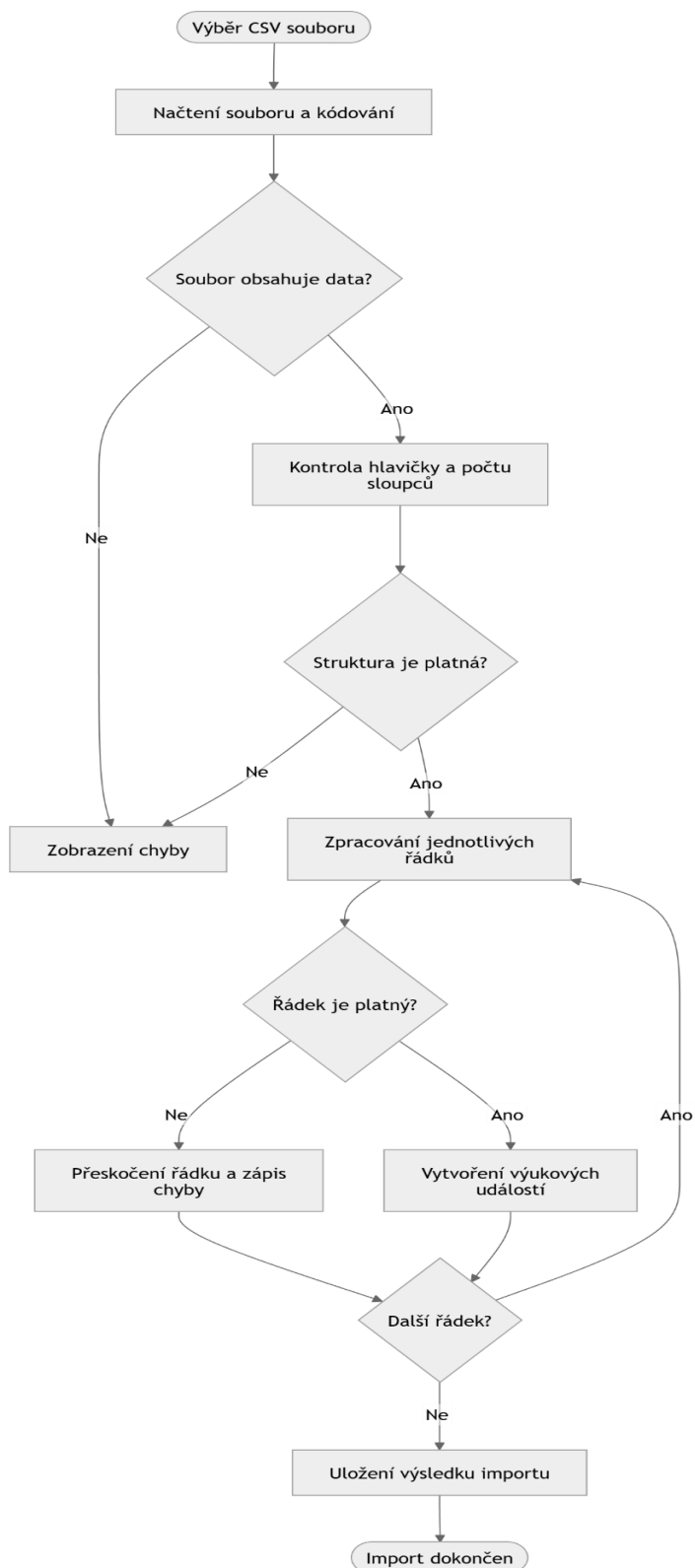


PŘÍLOHA L.3: Umístění a uložení obědové přestávky



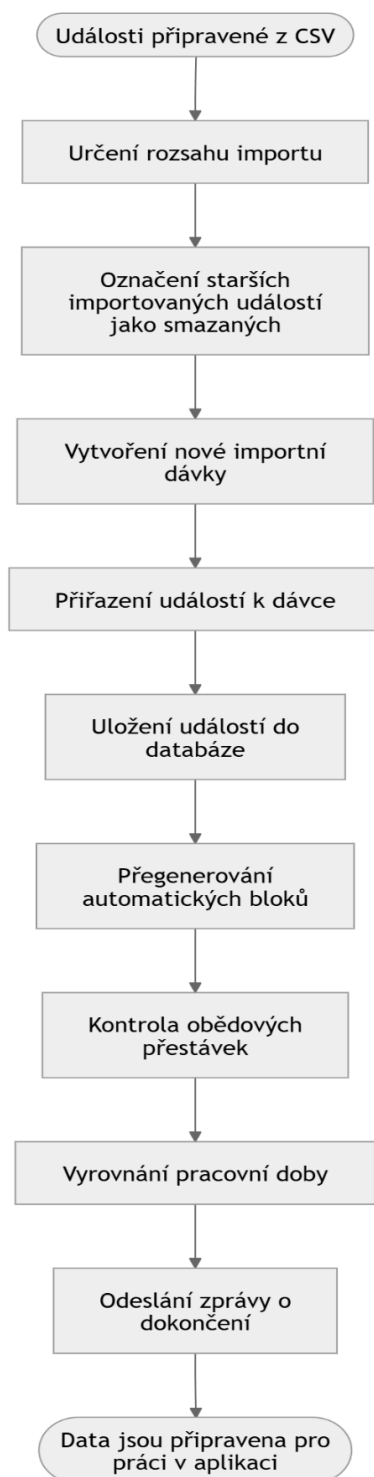
PŘÍLOHA M: Základní průběh načtení rozvrhu ze systému STAG

Tato příloha obsahuje blokové schéma základního průběhu načtení rozvrhu ze systému STAG. Schéma zachycuje načtení CSV souboru, kontrolu jeho struktury, zpracování jednotlivých řádků a vytvoření výukových událostí.



PŘÍLOHA N: Uložení načtených dat a navazující zpracování evidence

Tato příloha obsahuje blokové schéma navazujícího zpracování po načtení rozvrhových dat ze systému STAG. Schéma ukazuje uložení dávky načtených dat, označení starších rozvrhových událostí jako neaktivních, opětovné vytvoření automatických pracovních bloků, kontrolu obědových přestávek a spuštění vyrovnání pracovní doby.



PŘÍLOHA O: Souhrnná tabulka ověřených scénářů a řešených stavů

Tato příloha obsahuje souhrnnou tabulku ověřených scénářů a řešených stavů. Tabulka zachycuje hlavní testované oblasti aplikace, například načtení rozvrhu ze systému STAG, práci s událostmi, obědové přestávky, databázové ukládání, výstupní dokument PDF, uživatelské rozhraní a opakovatelnost zpracování.

ID	Oblast	Scénář	Popis problému	Řešení problému
1	Načtení rozvrhu	Načtení platného CSV souboru ze systému STAG	Je nutné ověřit, zda aplikace umí načíst běžný vstupní soubor a vytvořit z něj události.	Soubor byl načten, události byly vytvořeny podle rozvrhu a uloženy do databáze.
2	Načtení rozvrhu	Načtení prázdného CSV souboru	Soubor neobsahuje žádná data pro vytvoření událostí.	Načtení bylo odmítnuto a uživatel byl informován, že vstupní soubor neobsahuje data.
3	Načtení rozvrhu	CSV soubor s chybějícími nebo nedostatečnými sloupci	Struktura souboru neodpovídá očekávanému formátu.	Aplikace rozpoznala neplatnou strukturu a zabránila uložení nekorektních dat.
4	Načtení rozvrhu	CSV řádek s poškozenou strukturou nebo nepárovými uvozovkami	Jeden řádek nelze bezpečně rozpoznat jako platný záznam.	Problematický řádek byl vyhodnocen jako chybný a nebyl uložen jako rozvrhová událost.
5	Načtení rozvrhu	CSV řádek s neplatným datem začátku nebo konce období	Z řádku nelze určit platné kalendářní období.	Řádek nebyl použit pro vytvoření událostí a databáze zůstala konzistentní.
6	Načtení rozvrhu	CSV řádek s neplatným časem začátku nebo konce	Z řádku nelze vytvořit platný časový interval.	Řádek byl přeskočen a nebyl uložen jako událost.
7	Načtení rozvrhu	CSV řádek, ve kterém je konec události dříve než začátek	Časový interval události je logicky neplatný.	Aplikace řádek odmítla a nevytvořila chybný záznam.
8	Načtení rozvrhu	CSV řádek s neplatným rozsahem týdnů	Rozvrhový údaj neumožňuje určit týdně, ve kterých má událost vzniknout.	Řádek nebyl použit pro vytvoření událostí.
9	Opakované načtení rozvrhu	Opakované načtení stejného období	Při opakovaném načtení by mohly vzniknout duplicitní aktivní události.	Starší rozvrhové události v daném rozsahu byly označeny jako neaktivní a nahrazeny novou dávkou načtených dat.
10	Dávky načtených dat	Uložení informací o dávce načtených dat	Po načtení rozvrhu je nutné zpětně určit, které události patří ke konkrétní dávce.	Aplikace vytvořila záznam dávky načtených dat a nově vytvořené rozvrhové události k němu přiřadila.
11	Automatické pracovní bloky	Regenerace automatických pracovních bloků po importu	Po importu se může změnit denní rozvržení práce a volných úseků.	Aplikace přepočítala automaticky generované bloky pro dotčené období.
12	Vyrovnaní pracovní doby	Přepočet evidence po změně rozsahu událostí	Změna událostí může ovlivnit denní, týdenní i měsíční součty.	Aplikace obnovila automatické pracovní bloky a provedla návazné vyrovnaní pracovní doby.
13	Kolize událostí	Překryv dvou ručně zadaných nebo importovaných událostí ve stejném dni	Překrývající se události mohou zkreslit výslednou evidenci.	Události byly označeny jako kolizní, aby je mohl uživatel zkontrolovat.
14	Typy událostí	Zpracování pracovních událostí, výuky a pracovních cest	Různé pracovní typy událostí musí být zahrnuty do výpočtu pracovní doby.	Práce, výuka a pracovní cesta byly při výpočtech zohledněny jako pracovní čas.

ID	Oblast	Scénář	Popis problému	Řešení problému
15	Typy událostí	Zpracování nepřítomností, například nemoc, dovolená, OČR nebo lékař	Nepřítomnosti musí být odlišeny od běžné práce a správně zobrazeny ve výstupu.	Aplikace rozlišila typ nepřítomnosti a použila jej při evidenci i ve výsledném PDF.
16	Denní struktura	Pracovní den delší než 4 hodiny	Při delší pracovní době musí být vytvořena jedna přestávka.	Aplikace vytvořila jednu přestávku a rozdělila pracovní bloky tak, aby se s ní nepřekrývaly.
17	Denní struktura	Pracovní den delší než 12 hodin	Při pracovní době delší než 12 hodin musí být vytvořeny dvě přestávky.	Aplikace vytvořila dvě přestávky a upravila navazující pracovní bloky podle jejich umístění.
18	Denní struktura	Ruční vytvoření přestávky jako celodenní události	Přestávka nemůže platit pro celý kalendářní den.	Aplikace takový záznam odmítla.
19	Denní struktura	Ruční vytvoření přestávky přesahující jeden kalendářní den	Přestávka musí být evidována pouze v rámci jednoho dne.	Aplikace záznam odmítla.
20	Denní struktura	Ruční vytvoření přestávky s koncem před začátkem	Interval přestávky je časově neplatný.	Aplikace záznam neuložila.
21	Denní struktura	Ruční vytvoření přestávky delší než povolená maximální délka	Délka přestávky překračuje povolené nastavení dne.	Aplikace záznam odmítla nebo uživatele upozornila na překročení délky.
22	Denní struktura	Přestávka kolidující s jinou událostí	Přestávka se může překrývat s výukou nebo jinou ručně zadanou událostí.	Aplikace se pokusila přestávku přesunout do volného místa nebo označila kolizní stav.
23	Nastavení dne	Ruční úprava začátku nebo konce pracovního dne	Uživatel může změnit denní hranice a tím ovlivnit automatické bloky.	Aplikace uložila ruční nastavení a použila ho při dalším zpracování.
24	Nastavení dne	Změna nastavení pro konkrétní den	Jednotlivý den se může lišit od obecných semestrálních pravidel.	Aplikace uložila individuální nastavení dne a odlišila ho od výchozích pravidel.
25	Nastavení semestru	Změna výchozí pracovní doby v semestrálním nastavení	Globální nastavení ovlivňuje další generování pracovních dnů.	Nové nastavení bylo použito jako výchozí základ pro další zpracování.
26	Databáze	Uložení událostí, nastavení dne, nastavení semestru a importních dávek	Data musí být zachována i po ukončení aplikace.	Údaje byly uloženy v lokální databázi SQLite a byly dostupné při dalším spuštění.
27	Databáze	Propojení událostí s importní dávkou	U importovaných událostí musí být dohledatelné, z jakého importu pocházejí.	Údlosti byly propojeny se záznamem importní dávky.
28	Databáze	Práce s lokálním uživatelským záznamem	Evidence je vedena pro lokální profil bez samostatné registrace více uživatelů.	Aplikace pracovala s lokálním profilem osoby, pro kterou je evidence vedena.
29	Výstupní dokument PDF	Generování měsíčního přehledu ve formátu PDF	Uživatel potřebuje vytvořit výstup za zvolený měsíc.	Aplikace vytvořila PDF dokument s údaji za zvolený měsíc.
30	Výstupní dokument PDF	Zobrazení typů nepřítomností a pracovních cest ve výstupu	Ve výstupu musí být jasně rozlišeny jednotlivé typy událostí.	V dokumentu byly použity odpovídající zkratky pro dovolenou, nemoc, OČR, lékaře, pracovní cestu a svátek.
31	Výstupní dokument PDF	Kontrola součtů ve vygenerovaném přehledu	Chybné součty by vedly k nesprávné evidenci pracovní doby.	Součty ve výstupu odpovídaly zpracovaným událostem a pravidlům evidence.
32	Uživatelské rozhraní	Vytvoření nové události přes okno pro vytvoření události	Uživatel musí mít možnost ručně doplnit událost do evidence.	Údlost byla vytvořena se zadaným názvem, typem, datem a časem.
33	Uživatelské rozhraní	Úprava existující události přes okno pro změnu události	Změny události se musí projevit v zobrazení i ve výpočtech.	Upravené údaje byly uloženy a promítny se do kalendáře i výpočtů.

ID	Oblast	Scénář	Popis problému	Řešení problému
34	Uživatelské rozhraní	Odstranění existující události	Smazaná událost se nesmí dále používat v běžném zobrazení a výpočtech.	Událost byla odstraněna nebo označena jako smazaná.
35	Kalendářové zobrazení	Zobrazení událostí v denním pohledu	Uživatel potřebuje vidět detail jednoho dne.	Události byly zobrazeny podle svého časového rozsahu v rámci konkrétního dne.
36	Kalendářové zobrazení	Zobrazení událostí v týdenním pohledu	Uživatel potřebuje kontrolovat rozložení událostí v rámci týdne.	Události byly zobrazeny v týdenním kontextu.
37	Kalendářové zobrazení	Zobrazení událostí v měsíčním pohledu	Uživatel potřebuje širší přehled o evidenci v měsíci.	Události byly zobrazeny v měsíčním pohledu.
38	Chybové stavy	Pokus o zpracování neplatných vstupních dat	Neplatná data by mohla způsobit nekonzistentní stav evidence.	Aplikace neuložila nekonzistentní záznamy a informovala uživatele o problému.
39	Opakovatelnost zpracování	Opakované načtení stejného rozvrhu a následného zpracování	Opakovaný postup musí vést ke stejnému výsledku bez duplicit.	Aplikace vytvořila stejný výsledný stav evidence bez duplicitních aktivních rozvrhových událostí.